final-melhorado-v2

June 16, 2026

[1]:
```
'''
Bibliotecas usadas
'''

import os
from datetime import datetime
import re
import subprocess
import warnings
import numpy as np
import pandas as pd
from collections import Counter
from Bio import Entrez
from Bio import SeqIO
from Bio.Seq import Seq
from sklearn.model_selection import (
    GroupShuffleSplit,
    StratifiedGroupKFold,
    RandomizedSearchCV,
    GroupKFold
    )
from sklearn.metrics import (
    classification_report,
    roc_auc_score,
    average_precision_score,
    make_scorer,
    matthews_corrcoef,
    f1_score,
    precision_score,
    recall_score,
    balanced_accuracy_score,
    accuracy_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_curve,
    auc
)
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_validate
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from xgboost import XGBClassifier
import matplotlib.pyplot as plt
import subprocess
import shap
import seaborn as sns
from sklearn.inspection import permutation_importance


warnings.filterwarnings("ignore")

print("Bibliotecas importadas")
```

```
Bibliotecas importadas
```

[2]:
```
'''
obtenção dos datasets
'''

EMAIL = "marcela.leite@gmail.com.br"
OUTPUT_DIR = "pipelineAG"
os.makedirs(OUTPUT_DIR, exist_ok=True)
Entrez.email = EMAIL

# ============================================================
# QUERIES
# ============================================================

POSITIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum betaceum"[Organism] OR "Solanum␣
 ↪melongena"[Organism] OR "Solanum tuberosum"[Organism] OR "Physalis␣
 ↪peruviana"[Organism]))
    AND (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA␣
 ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "Rx1" OR "Sw-5" OR "Tm-2" OR "Tm-22" OR "RCY1" OR "Rx" OR "N GENE"
    OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR" OR "CC-NBS-LRR" OR "TNL"␣
 ↪OR "CNL Domain" OR "CNL"
    OR "NBS-ARC" OR "NB-ARC domain" OR "NB-ARC" OR "Nucleotide-binding site"
    OR "RDR" OR "Dicer-like" OR "Argonaute" OR "AGO" OR "eIF4E" )
    NOT (partial OR fragment OR hypothetical OR predicted OR DAP-seq OR␣
 ↪"transcription factor" OR DNA-binding OR microtom)
```

```
    NOT ("pathogenesis-reletad protein" OR osmotin OR PR1 OR PR2 OR PR3 OR PR4␣
 ↪OR PR5 OR synthase OR metabolic OR precursor)
    NOT txid10239[Organism:exp]
'''
NEGATIVE_QUERY = r'''
    ("Solanaceae"[Organism] NOT ("Solanum melongena"[Organism] OR "Solanum␣
 ↪tuberosum"[Organism] OR "Physalis peruviana"[Organism]))
   AND 100:3000[Sequence Length]
   NOT (TMV OR ToMV OR ToBRFV OR PMMoV OR TMGMV OR PVY OR PepYMV OR TEV OR PVA␣
 ↪OR PVX OR TSWV OR GRSV OR TCSV OR CMV OR ToCV OR TICV OR AMV
    OR "Rx1" OR "Sw-5" OR "Tm-2" OR "Tm-22" OR "RCY1" OR "Rx" OR "N"
    OR "virus resistance" OR "antiviral" OR "Tobacco mosaic virus" OR "Potato␣
 ↪virus"
    OR "NLR" OR "NBS-LRR" OR "NB-LRR" OR "TIR-NBS-LRR" OR "CC-NBS-LRR" OR "TNL"␣
 ↪OR "CNL Domain" OR "CNL"
    OR "NBS-ARC" OR "NB-ARC domain" OR "NB-ARC" OR "Nucleotide-binding site"
    OR "RNA silencing" OR "RDR" OR "Dicer-like" OR "Argonaute" OR "AGO" OR␣
 ↪"eIF4E" )
   NOT (partial OR fragment OR hypothetical OR predicted OR uncharacterized OR␣
 ↪DAP-seq OR "transcription factor" OR DNA-binding OR microtom )
'''

ARABIDOPSIS_QUERY = '''
   "Arabidopsis thaliana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
'''

SOLANUM_MELONGENA = '''
    "Solanum melongena"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
    '''

TUBEROSUM_QUERY = '''
    "Solanum tuberosum"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
    '''

PHYSALIS_QUERY = '''
  "Physalis peruviana"[Organism] AND (biomol_mrna[PROP] OR tsa[keyword])
      AND 1500:30000[Sequence Length]
      '''

# ============================================================
# DOWNLOAD NCBI
# ============================================================
```

```python
def baixar_proteinas_ncbi(query, output_fasta, max_records=35000):
    print("\n================================")
    print("DOWNLOAD NCBI: ",output_fasta)
    print("================================")
    handle = Entrez.esearch(
        db="protein",
        term=query,
        retmax=max_records
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Proteínas encontradas: {len(ids)}")
    if len(ids) == 0:
        return
    handle = Entrez.efetch(
        db="protein",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

def baixar_transcriptoma(output_fasta, query):
    print("\n================================")
    print("DOWNLOAD TRANSCRIPTOMA: ",output_fasta)
    print("================================")
    handle = Entrez.esearch(
        db="nucleotide",
        term=query,
        retmax=50000
    )
    record = Entrez.read(handle)
    ids = record["IdList"]
    print(f"Transcritos encontrados: {len(ids)}")
    handle = Entrez.efetch(
        db="nucleotide",
        id=ids,
        rettype="fasta",
        retmode="text"
    )
    with open(output_fasta, "w") as f:
        f.write(handle.read())
    print(f"Arquivo salvo: {output_fasta}")

#Dados de Entrada - baixar arquivos FASTA das sequências de proteínas
```

```
POSITIVE_FASTA = f"{OUTPUT_DIR}/positive.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative.fasta"


print("\n============ FINALIZADO CONFIGURAÇÃO DE PARÂMETROS ============\n")
```

```
============ FINALIZADO CONFIGURAÇÃO DE PARÂMETROS ============
```

[3]:
```
inicio = datetime.now()
print("\n\nInciando download da classe positiva: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(POSITIVE_QUERY,POSITIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download da classe positiva:  10:32:10

================================
DOWNLOAD NCBI:  pipelineAG/positive.fasta
================================
Proteínas encontradas: 8914
Arquivo salvo: pipelineAG/positive.fasta


Finalizado em: 10:33:03

Tempo decorrido: 0:00:53.013381
```

[4]:
```
inicio = datetime.now()
print("\n\nInciando download da classe negativa: ", inicio.strftime("%H:%M:%S"))
baixar_proteinas_ncbi(NEGATIVE_QUERY,NEGATIVE_FASTA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\n\nArquivos salvos na pasta: ",OUTPUT_DIR)
print("============ FIM COLETA DE DADOS TREINO/VALIDAÇÃO ============")
```

```
Inciando download da classe negativa:  10:33:03

================================
DOWNLOAD NCBI:  pipelineAG/negative.fasta
================================
```

```
Proteínas encontradas: 35000
Arquivo salvo: pipelineAG/negative.fasta


Finalizado em: 10:34:03

Tempo decorrido: 0:01:00.093673


Arquivos salvos na pasta:  pipelineAG
============ FIM COLETA DE DADOS TREINO/VALIDAÇÃO ============
```

[5]:
```
#Dados para aplicação - baixar transcriptomas - RNA-Seq
transcriptoma_arabidopsis_fasta = (f"{OUTPUT_DIR}/arabidopsis_transcriptoma.
 ↪fasta")
transcriptoma_melongena_fasta = (f"{OUTPUT_DIR}/melongena_transcriptoma.fasta")
transcriptoma_tuberosum_fasta = (f"{OUTPUT_DIR}/tuberosum_transcriptoma.fasta")
transcriptoma_physalis_fasta = (f"{OUTPUT_DIR}/physalis_transcriptoma.fasta")
```

[6]:
```
print("\n\nInciando downaload de dados de transcriptmas")
inicio = datetime.now()
print("\n\nInciando download ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_arabidopsis_fasta, ARABIDOPSIS_QUERY)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando downaload de dados de transcriptmas


Inciando download ARABIDOPSIS:  10:34:03

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAG/arabidopsis_transcriptoma.fasta
================================
Transcritos encontrados: 49951
Arquivo salvo: pipelineAG/arabidopsis_transcriptoma.fasta


Finalizado em: 10:34:46

Tempo decorrido: 0:00:43.022369
```

[8]:
```
inicio = datetime.now()
print("\n\nInciando download PHYSALIS: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_physalis_fasta, PHYSALIS_QUERY)
```

```
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download PHYSALIS:  10:35:39

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAG/physalis_transcriptoma.fasta
================================
Transcritos encontrados: 21702
Arquivo salvo: pipelineAG/physalis_transcriptoma.fasta


Finalizado em: 10:36:21

Tempo decorrido: 0:00:42.376873
```

[9]:
```
inicio = datetime.now()
print("\n\nInciando download MELONGENA: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_melongena_fasta, SOLANUM_MELONGENA)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Inciando download MELONGENA:  10:36:21

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAG/melongena_transcriptoma.fasta
================================
Transcritos encontrados: 8555
Arquivo salvo: pipelineAG/melongena_transcriptoma.fasta


Finalizado em: 10:36:59

Tempo decorrido: 0:00:38.041127
```

[10]:
```
inicio = datetime.now()
print("\n\nInciando download TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
baixar_transcriptoma(transcriptoma_tuberosum_fasta, TUBEROSUM_QUERY)
fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
print("\n\n\nArquivos salvos na pasta: ",OUTPUT_DIR)
print("============ FIM COLETA DE DADOS APLICAÇÃO ============")
```

```
Inciando download TUBEROSUM:  10:36:59

================================
DOWNLOAD TRANSCRIPTOMA:  pipelineAG/tuberosum_transcriptoma.fasta
================================
Transcritos encontrados: 20832
Arquivo salvo: pipelineAG/tuberosum_transcriptoma.fasta


Finalizado em: 10:37:42

Tempo decorrido: 0:00:43.120825



Arquivos salvos na pasta:  pipelineAG
============ FIM COLETA DE DADOS APLICAÇÃO ============
```

[11]:
```
'''
Tratamento dos dados
    - Criar Dataframes com os conjuntos de dados de treino e validação
    - Executar CD-Hit para eliminar sequencias muitos semelhantes
    - Extração de atributos (features)

'''

# ============================================================
# AAC
# ============================================================
# composição de aminoácidos - transformar a proteína em dados numéricos
# conta a frequência de cada aminoácido em uma sequência de proteína
# como são 20 aminoácidos possíveis, irá gerar 20 features com seus percentuais
# com essa frequência consegue ter um parâmetro para distinguir as proteínas
# por exemplo proteínas de resistência tem geralmente certa frequência de␣
 ↪aminoácidos
# mas ignora a ordem - por isso a sequência de dipeptídeos irá incrementar o␣
 ↪modelo considerando a ordem
# sequência de 20 aminoácidos
AMINO_ACIDOS = list("ACDEFGHIKLMNPQRSTVWY")

motifs = {
    # ==========================
```

```
    # NLR (CNL/TNL)
    # ==========================
    "P_LOOP": r"[AGST].{3,5}GK[TS]",
    "KINASE2": r"[ALIVMFYW]{3,5}DD[VILW][WFY]",
    "GLPL": r"G[LVI][PAST][LIVAF]",
    "MHD": r"[MVIL][HN][DE][VILAST]",
    "RNBS_A": r"F.DL.{0,3}AW",
    "RNBS_B": r"G[SA].{2,6}T",
    "RNBS_C": r"Y.[VIL].{1,3}[LS]",
    "RNBS_D": r"C.{2,8}P",

    # ==========================
    # Protein Kinase (KIN/RLK)
    # ==========================
    "HRD": r"HRD[LIVMFY]",
    "DFG": r"DFG",

    # ==========================
    # LRR (RLP/RLK)
    # ==========================
    "LRR1": r"L..L..L..N",
    "LRR2": r"L[A-Z]{2}L[A-Z]{2}L[A-Z]{2}[NCT]"

}

def calcular_aac(seq):
    seq = seq.upper()
    total = len(seq)
    if total == 0:
        return [0] * len(AMINO_ACIDOS)
    counts = Counter(seq)
    return [
        counts.get(aa, 0) / total
        for aa in AMINO_ACIDOS
    ]

# ============================================================
# DIPEPTÍDEOS
# ============================================================
'''
encontrar pares de aminoácidos consectíveis em uma sequência protéica ␣
 ↪-sequência de dois aminoácidos
considera a ordem dos aminoácidos
há certos padrões comuns para
para 20 aminoácidos, há possibilidade de 20*20 = 400 dipeptídeos - features
'''
```

```
DIPEPTIDES = [
    a + b
    for a in AMINO_ACIDOS
    for b in AMINO_ACIDOS
]

def calcular_dipeptideos(seq):
    seq = seq.upper()
    total = len(seq) - 1
    if total <= 0:
        return [0] * len(DIPEPTIDES)
    counts = Counter([
        seq[i:i+2]
        for i in range(total)
    ])
    return [
        counts.get(dp, 0) / total
        for dp in DIPEPTIDES
    ]

def localizar_motivos(seq):
    posicoes = {}
    for nome, pattern in motifs.items():
        match = re.search(pattern, seq)

        if match:
            posicoes[nome] = match.start()
        else:
            posicoes[nome] = None

    return posicoes


def calcular_distancias(posicoes):

    def distancia(a, b):
        if posicoes[a] is None or posicoes[b] is None:
            return -1
        return posicoes[b] - posicoes[a]

    return {
        "dist_ploop_kinase2": distancia("P_LOOP", "KINASE2"),
        "dist_kinase2_glpl": distancia("KINASE2", "GLPL"),
        "dist_glpl_mhd": distancia("GLPL", "MHD")
    }

def contar_motivos(posicoes):
```

```
    return sum(
        1 for v in posicoes.values()
        if v is not None
    )

def extrair_motifs(seq):

    return [
        int(bool(re.search(pattern, seq)))
        for pattern in motifs.values()
    ]



# ============================================================
# FEATURES - atributos
# ============================================================
#calculando features
def extrair_features_proteina(seq):
    seq = re.sub(r"[^ACDEFGHIKLMNPQRSTVWY]", "", seq)
    if len(seq) < 50:
        return None
    features = []
    features.append(len(seq)) # 1 - comprimento da cadeia
    features.extend(calcular_aac(seq)) # 20 composição de aminoácidos - frequência de cada um na sequência
    features.extend(calcular_dipeptideos(seq)) #400 possibilidades - considera a ordem dos aminoácidos para buscar os mais frequêntes em proteínas de resistência

    features.extend(extrair_motifs(seq))
    # posições dos motivos
    posicoes = localizar_motivos(seq)
    # número de motivos encontrados
    num_motifs_detectados = contar_motivos(posicoes)
    # distâncias entre motivos
    distancias = calcular_distancias(posicoes)

    features.append(num_motifs_detectados)
    protein_length = len(seq)

    features.append(distancias["dist_ploop_kinase2"])
    features.append(distancias["dist_kinase2_glpl"])
    features.append(distancias["dist_glpl_mhd"])

    features.append(
        distancias["dist_ploop_kinase2"]/protein_length
```

```
        if distancias["dist_ploop_kinase2"] != -1 else -1
    )
    features.append(
        distancias["dist_kinase2_glpl"]/protein_length
        if distancias["dist_kinase2_glpl"] != -1 else -1
    )
    features.append(
        distancias["dist_glpl_mhd"]/protein_length
        if distancias["dist_glpl_mhd"] != -1 else -1
    )

    features.append(int(posicoes["P_LOOP"] is not None))
    features.append(int(posicoes["KINASE2"] is not None))
    features.append(int(posicoes["GLPL"] is not None))
    features.append(int(posicoes["MHD"] is not None))

    encontrados = {}
    for nome, regex in motifs.items():
        encontrados[nome] = int( re.search(regex, seq) is not None)

    # features.append(encontrados[nome])

    num_nlr = (
        encontrados["P_LOOP"] +
        encontrados["KINASE2"] +
        encontrados["GLPL"] +
        encontrados["MHD"]
    )

    # num_kinase = (
    #     encontrados["HRD"] +
    #     encontrados["DFG"]
    # )

    # num_lrr = (
    #     encontrados["LRR1"] +
    #     encontrados["LRR2"]
    # )

    features.extend([
        num_nlr,
        # num_kinase,
        # num_lrr
    ])

    return features
```

```
# ============================================================
# DATASET
# ============================================================
motif_cols = [
    "P_LOOP",
    "KINASE2",
    "GLPL",
    "MHD",
    "RNBS_A",
    "RNBS_B",
    "RNBS_C",
    "RNBS_D",
    "HRD",
    "DFG",
    "LRR1",
    "LRR2",
    "NUM_NLR_MOTIFS"
 ]
    # "NUM_KINASE_MOTIFS",
    # "NUM_LRR_MOTIFS"

def criar_dataset(positive_fasta, negative_fasta):
    data = []
    columns = (
        ["protein_length"]
        + [f"AAC_{aa}" for aa in AMINO_ACIDOS]
        + [f"DIPEP_{dp}" for dp in DIPEPTIDES]
        + motif_cols
        + ["num_motifs_detectados"]
        + ["dist_ploop_kinase2"]
        + ["dist_kinase2_glpl"]
        + ["dist_glpl_mhd"]
        + ["dist_ploop_kinase2_norm"]
        + ["dist_kinase2_glpl_norm"]
        + ["dist_glpl_mhd_norm"]
        + ["has_ploop"]
        + ["has_kinase2"]
        + ["has_glpl"]
        + ["has_mhd"]
    )

    # POSITIVOS
    for record in SeqIO.parse(positive_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
```

```
        row = feats + [1, record.id]
        data.append(row)

    # NEGATIVOS
    for record in SeqIO.parse(negative_fasta, "fasta"):
        seq = str(record.seq)
        feats = extrair_features_proteina(seq)
        if feats is None:
            continue
        row = feats + [0, record.id]
        data.append(row)

    df = pd.DataFrame(
        data,
        columns=columns + ["target", "id"]
    )
    print("\nTotal de Features/Atributos:",len(columns)) # quantidade
    print("\nFeatures/atributos:")
    print(columns)
    return df, columns

# CD-Hit
# chama o programa CD-HIT do sistema operacional
def executar_cdhit(
    input_fasta,
    output_fasta,
    identity=0.7
):
    cmd = [
        "cd-hit",
        "-i", input_fasta,
        "-o", output_fasta,
        "-c", str(identity),
        "-n", "5",
        "-M", "30000",
        "-T", "4"
    ]

    print("\nExecutando CD-HIT...")
    subprocess.run(cmd, check=True)
    print("CD-HIT concluído.")

def limpar(seq_id):
    seq_id = seq_id.strip()
    seq_id = seq_id.split()[0]
    return seq_id
```

```
def ler_clusters_cdhit(cluster_file):
    clusters = {}
    cluster_id = None
    with open(cluster_file) as f:
        for line in f:
            line = line.strip()
            if line.startswith(">Cluster"):
                cluster_id = line.replace(">Cluster ", "")
                clusters[cluster_id] = []
            else:
                seq_id = line.split(">")[1].split("...")[0]
                seq_id = limpar(seq_id)
                clusters[cluster_id].append(seq_id)
    return clusters

inicio = datetime.now()
print("\n\n\nInciando Tratamento dos dados: ", inicio.strftime("%H:%M:%S"))

# executa o CD-Hit nos arquivos baixados
print("\n\nInciar a execução do CD-HIT\n")
executar_cdhit(
    POSITIVE_FASTA,
    f"{OUTPUT_DIR}/positive_nr.fasta",
    identity=0.7
)

executar_cdhit(
    NEGATIVE_FASTA,
    f"{OUTPUT_DIR}/negative_nr.fasta",
    identity=0.7
)
# ========================================================
# Montar os DATASETs
# ========================================================

POSITIVE_FASTA = f"{OUTPUT_DIR}/positive_nr.fasta"
NEGATIVE_FASTA = f"{OUTPUT_DIR}/negative_nr.fasta"


df, columns = criar_dataset(
    POSITIVE_FASTA,
    NEGATIVE_FASTA
)

df["id"] = df["id"].apply(limpar)
# ========================================================
# CLUSTERS
```

```
# ======================================================

clusters_pos = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/positive_nr.fasta.clstr"
)

clusters_neg = ler_clusters_cdhit(
    f"{OUTPUT_DIR}/negative_nr.fasta.clstr"
)

cluster_map = {}

for c, seqs in clusters_pos.items():
    for s in seqs:
        cluster_map[s] = f"POS_{c}"

for c, seqs in clusters_neg.items():
    for s in seqs:
        cluster_map[s] = f"NEG_{c}"

df["cluster"] = df["id"].map(cluster_map)

print("\n Clusters ausentes: ")
print(df["cluster"].isna().sum())
df = df.dropna(subset=["cluster"])

print("\nDataset original pós CD-HIT:", df.shape)

# ======================================================
# NOVO BLOCO: BALANCEAMENTO IMPERATIVO (DOWNSAMPLING)
# ======================================================
print("\n--- Executando Balanceamento de Classes ---")
df_positivos = df[df["target"] == 1]
df_negativos = df[df["target"] == 0]

print(f"Contagem inicial -> Positivos (R-genes): {len(df_positivos)} |␣
 ↪Negativos (Background): {len(df_negativos)}")

# Se houver mais negativos do que positivos (cenário padrão), reduzimos os␣
 ↪negativos
if len(df_negativos) > len(df_positivos):
    # O sample() escolhe aleatoriamente N linhas do grupo majoritário
    df_negativos_balanceados = df_negativos.sample(n=len(df_positivos),␣
 ↪random_state=42)
    df = pd.concat([df_positivos, df_negativos_balanceados]).
 ↪reset_index(drop=True)
    print(f"Subamostragem aplicada com sucesso nos Negativos.")
```

```
else:
    print("O dataset já está balanceado ou possui mais positivos que negativos␣
 ↪(incomum). Nenhuma ação foi tomada.")

print(f"Contagem final   -> Positivos: {len(df[df['target'] == 1])} | Negativos:
 ↪ {len(df[df['target'] == 0])}")
print("Dataset Final Pronto:", df.shape)


fim = datetime.now()
print("\n\nFinalizado em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
print("\n\nFim tratamento dos dados...")
```

```
Inciando Tratamento dos dados:  10:37:42


Inciar a execução do CD-HIT


Executando CD-HIT…
================================================================
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAG/positive.fasta -o
         pipelineAG/positive_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

Started: Tue Jun 16 10:37:42 2026
================================================================
                            Output
----------------------------------------------------------------
total seq: 8914
longest and shortest : 3540 and 51
Total letters: 7801441
Sequences have been sorted

Approximated minimal memory consumption:
Sequence        : 8M
Buffer          : 4 X 11M = 45M
Table           : 2 X 65M = 130M
Miscellaneous   : 0M
Total           : 185M

Table limit with the given memory limit:
Max number of representatives: 4000000
```

```
Max number of word counting entries: 3726820522

# comparing sequences from          0  to       1485
.---------- new table with      236 representatives
# comparing sequences from       1485  to       2723
99.9%---------- new table with      192 representatives
# comparing sequences from       2723  to       3754
----------     84 remaining sequences to the next cycle
---------- new table with      166 representatives
# comparing sequences from       3670  to       4544
99.9%---------- new table with      123 representatives
# comparing sequences from       4544  to       5272
100.0%---------- new table with       63 representatives
# comparing sequences from       5272  to       5879
…---------- new table with       87 representatives
# comparing sequences from       5879  to       6384
----------    109 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       6275  to       6714
----------     65 remaining sequences to the next cycle
---------- new table with      118 representatives
# comparing sequences from       6649  to       7026
----------     57 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       6969  to       7293
----------     59 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       7234  to       7514
…---------- new table with       76 representatives
# comparing sequences from       7514  to       7747
…---------- new table with       64 representatives
# comparing sequences from       7747  to       7941
…---------- new table with       48 representatives
# comparing sequences from       7941  to       8914
…---------- new table with      246 representatives

     8914  finished       1719  clusters

Approximated maximum memory consumption: 190M
writing new database
writing clustering information
program completed !

Total CPU time 3.60
CD-HIT concluído.

Executando CD-HIT…
================================================================
```

```
Program: CD-HIT, V4.8.1 (+OpenMP), Aug 20 2021, 08:39:56
Command: cd-hit -i pipelineAG/negative.fasta -o
         pipelineAG/negative_nr.fasta -c 0.7 -n 5 -M 30000 -T 4

Started: Tue Jun 16 10:37:43 2026
================================================================
                            Output
----------------------------------------------------------------
total seq: 9999
longest and shortest : 2993 and 100
Total letters: 4204463
Sequences have been sorted

Approximated minimal memory consumption:
Sequence        : 5M
Buffer          : 4 X 11M = 44M
Table           : 2 X 65M = 131M
Miscellaneous   : 0M
Total           : 181M

Table limit with the given memory limit:
Max number of representatives: 4000000
Max number of word counting entries: 3727298858

# comparing sequences from          0  to       1666
.---------- new table with     1374 representatives
# comparing sequences from       1666  to       3054
----------    659 remaining sequences to the next cycle
---------- new table with      600 representatives
# comparing sequences from       2395  to       3662
----------    812 remaining sequences to the next cycle
---------- new table with      286 representatives
# comparing sequences from       2850  to       4041
----------    833 remaining sequences to the next cycle
---------- new table with      239 representatives
# comparing sequences from       3208  to       4339
----------    792 remaining sequences to the next cycle
---------- new table with      280 representatives
# comparing sequences from       3547  to       4622
----------    753 remaining sequences to the next cycle
---------- new table with      273 representatives
# comparing sequences from       3869  to       4890
----------    699 remaining sequences to the next cycle
---------- new table with      255 representatives
# comparing sequences from       4191  to       5159
----------    683 remaining sequences to the next cycle
---------- new table with      201 representatives
# comparing sequences from       4476  to       5396
```

```
----------    646 remaining sequences to the next cycle
---------- new table with      235 representatives
# comparing sequences from       4750  to       5624
----------    571 remaining sequences to the next cycle
---------- new table with      199 representatives
# comparing sequences from       5053  to       5877
----------    557 remaining sequences to the next cycle
---------- new table with      204 representatives
# comparing sequences from       5320  to       6099
----------    553 remaining sequences to the next cycle
---------- new table with      180 representatives
# comparing sequences from       5546  to       6288
----------    505 remaining sequences to the next cycle
---------- new table with      156 representatives
# comparing sequences from       5783  to       6485
----------    477 remaining sequences to the next cycle
---------- new table with      172 representatives
# comparing sequences from       6008  to       6673
----------    449 remaining sequences to the next cycle
---------- new table with      163 representatives
# comparing sequences from       6224  to       6853
----------    441 remaining sequences to the next cycle
---------- new table with      128 representatives
# comparing sequences from       6412  to       7009
----------    405 remaining sequences to the next cycle
---------- new table with      129 representatives
# comparing sequences from       6604  to       7169
----------    393 remaining sequences to the next cycle
---------- new table with      131 representatives
# comparing sequences from       6776  to       7313
----------    345 remaining sequences to the next cycle
---------- new table with      119 representatives
# comparing sequences from       6968  to       7473
----------    332 remaining sequences to the next cycle
---------- new table with      116 representatives
# comparing sequences from       7141  to       7617
----------    288 remaining sequences to the next cycle
---------- new table with      134 representatives
# comparing sequences from       7329  to       7774
----------    274 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       7500  to       7916
----------    267 remaining sequences to the next cycle
---------- new table with      117 representatives
# comparing sequences from       7649  to       8040
----------    199 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       7841  to       8200
```

```
----------    235 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       7965  to       8304
----------    189 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8115  to       8429
----------    163 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8266  to       8554
----------    135 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8419  to       8682
----------     83 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8599  to       8832
----------     36 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8796  to       8996
----------     41 remaining sequences to the next cycle
---------- new table with      100 representatives
# comparing sequences from       8955  to       9129
…---------- new table with       90 representatives
# comparing sequences from       9129  to       9999
…---------- new table with      412 representatives

     9999  finished       7093  clusters

Approximated maximum memory consumption: 194M
writing new database
writing clustering information
program completed !

Total CPU time 1.67
CD-HIT concluído.

Total de Features/Atributos: 445

Features/atributos:
['protein_length', 'AAC_A', 'AAC_C', 'AAC_D', 'AAC_E', 'AAC_F', 'AAC_G',
'AAC_H', 'AAC_I', 'AAC_K', 'AAC_L', 'AAC_M', 'AAC_N', 'AAC_P', 'AAC_Q', 'AAC_R',
'AAC_S', 'AAC_T', 'AAC_V', 'AAC_W', 'AAC_Y', 'DIPEP_AA', 'DIPEP_AC', 'DIPEP_AD',
'DIPEP_AE', 'DIPEP_AF', 'DIPEP_AG', 'DIPEP_AH', 'DIPEP_AI', 'DIPEP_AK',
'DIPEP_AL', 'DIPEP_AM', 'DIPEP_AN', 'DIPEP_AP', 'DIPEP_AQ', 'DIPEP_AR',
'DIPEP_AS', 'DIPEP_AT', 'DIPEP_AV', 'DIPEP_AW', 'DIPEP_AY', 'DIPEP_CA',
'DIPEP_CC', 'DIPEP_CD', 'DIPEP_CE', 'DIPEP_CF', 'DIPEP_CG', 'DIPEP_CH',
'DIPEP_CI', 'DIPEP_CK', 'DIPEP_CL', 'DIPEP_CM', 'DIPEP_CN', 'DIPEP_CP',
'DIPEP_CQ', 'DIPEP_CR', 'DIPEP_CS', 'DIPEP_CT', 'DIPEP_CV', 'DIPEP_CW',
'DIPEP_CY', 'DIPEP_DA', 'DIPEP_DC', 'DIPEP_DD', 'DIPEP_DE', 'DIPEP_DF',
```

'DIPEP_DG', 'DIPEP_DH', 'DIPEP_DI', 'DIPEP_DK', 'DIPEP_DL', 'DIPEP_DM',
'DIPEP_DN', 'DIPEP_DP', 'DIPEP_DQ', 'DIPEP_DR', 'DIPEP_DS', 'DIPEP_DT',
'DIPEP_DV', 'DIPEP_DW', 'DIPEP_DY', 'DIPEP_EA', 'DIPEP_EC', 'DIPEP_ED',
'DIPEP_EE', 'DIPEP_EF', 'DIPEP_EG', 'DIPEP_EH', 'DIPEP_EI', 'DIPEP_EK',
'DIPEP_EL', 'DIPEP_EM', 'DIPEP_EN', 'DIPEP_EP', 'DIPEP_EQ', 'DIPEP_ER',
'DIPEP_ES', 'DIPEP_ET', 'DIPEP_EV', 'DIPEP_EW', 'DIPEP_EY', 'DIPEP_FA',
'DIPEP_FC', 'DIPEP_FD', 'DIPEP_FE', 'DIPEP_FF', 'DIPEP_FG', 'DIPEP_FH',
'DIPEP_FI', 'DIPEP_FK', 'DIPEP_FL', 'DIPEP_FM', 'DIPEP_FN', 'DIPEP_FP',
'DIPEP_FQ', 'DIPEP_FR', 'DIPEP_FS', 'DIPEP_FT', 'DIPEP_FV', 'DIPEP_FW',
'DIPEP_FY', 'DIPEP_GA', 'DIPEP_GC', 'DIPEP_GD', 'DIPEP_GE', 'DIPEP_GF',
'DIPEP_GG', 'DIPEP_GH', 'DIPEP_GI', 'DIPEP_GK', 'DIPEP_GL', 'DIPEP_GM',
'DIPEP_GN', 'DIPEP_GP', 'DIPEP_GQ', 'DIPEP_GR', 'DIPEP_GS', 'DIPEP_GT',
'DIPEP_GV', 'DIPEP_GW', 'DIPEP_GY', 'DIPEP_HA', 'DIPEP_HC', 'DIPEP_HD',
'DIPEP_HE', 'DIPEP_HF', 'DIPEP_HG', 'DIPEP_HH', 'DIPEP_HI', 'DIPEP_HK',
'DIPEP_HL', 'DIPEP_HM', 'DIPEP_HN', 'DIPEP_HP', 'DIPEP_HQ', 'DIPEP_HR',
'DIPEP_HS', 'DIPEP_HT', 'DIPEP_HV', 'DIPEP_HW', 'DIPEP_HY', 'DIPEP_IA',
'DIPEP_IC', 'DIPEP_ID', 'DIPEP_IE', 'DIPEP_IF', 'DIPEP_IG', 'DIPEP_IH',
'DIPEP_II', 'DIPEP_IK', 'DIPEP_IL', 'DIPEP_IM', 'DIPEP_IN', 'DIPEP_IP',
'DIPEP_IQ', 'DIPEP_IR', 'DIPEP_IS', 'DIPEP_IT', 'DIPEP_IV', 'DIPEP_IW',
'DIPEP_IY', 'DIPEP_KA', 'DIPEP_KC', 'DIPEP_KD', 'DIPEP_KE', 'DIPEP_KF',
'DIPEP_KG', 'DIPEP_KH', 'DIPEP_KI', 'DIPEP_KK', 'DIPEP_KL', 'DIPEP_KM',
'DIPEP_KN', 'DIPEP_KP', 'DIPEP_KQ', 'DIPEP_KR', 'DIPEP_KS', 'DIPEP_KT',
'DIPEP_KV', 'DIPEP_KW', 'DIPEP_KY', 'DIPEP_LA', 'DIPEP_LC', 'DIPEP_LD',
'DIPEP_LE', 'DIPEP_LF', 'DIPEP_LG', 'DIPEP_LH', 'DIPEP_LI', 'DIPEP_LK',
'DIPEP_LL', 'DIPEP_LM', 'DIPEP_LN', 'DIPEP_LP', 'DIPEP_LQ', 'DIPEP_LR',
'DIPEP_LS', 'DIPEP_LT', 'DIPEP_LV', 'DIPEP_LW', 'DIPEP_LY', 'DIPEP_MA',
'DIPEP_MC', 'DIPEP_MD', 'DIPEP_ME', 'DIPEP_MF', 'DIPEP_MG', 'DIPEP_MH',
'DIPEP_MI', 'DIPEP_MK', 'DIPEP_ML', 'DIPEP_MM', 'DIPEP_MN', 'DIPEP_MP',
'DIPEP_MQ', 'DIPEP_MR', 'DIPEP_MS', 'DIPEP_MT', 'DIPEP_MV', 'DIPEP_MW',
'DIPEP_MY', 'DIPEP_NA', 'DIPEP_NC', 'DIPEP_ND', 'DIPEP_NE', 'DIPEP_NF',
'DIPEP_NG', 'DIPEP_NH', 'DIPEP_NI', 'DIPEP_NK', 'DIPEP_NL', 'DIPEP_NM',
'DIPEP_NN', 'DIPEP_NP', 'DIPEP_NQ', 'DIPEP_NR', 'DIPEP_NS', 'DIPEP_NT',
'DIPEP_NV', 'DIPEP_NW', 'DIPEP_NY', 'DIPEP_PA', 'DIPEP_PC', 'DIPEP_PD',
'DIPEP_PE', 'DIPEP_PF', 'DIPEP_PG', 'DIPEP_PH', 'DIPEP_PI', 'DIPEP_PK',
'DIPEP_PL', 'DIPEP_PM', 'DIPEP_PN', 'DIPEP_PP', 'DIPEP_PQ', 'DIPEP_PR',
'DIPEP_PS', 'DIPEP_PT', 'DIPEP_PV', 'DIPEP_PW', 'DIPEP_PY', 'DIPEP_QA',
'DIPEP_QC', 'DIPEP_QD', 'DIPEP_QE', 'DIPEP_QF', 'DIPEP_QG', 'DIPEP_QH',
'DIPEP_QI', 'DIPEP_QK', 'DIPEP_QL', 'DIPEP_QM', 'DIPEP_QN', 'DIPEP_QP',
'DIPEP_QQ', 'DIPEP_QR', 'DIPEP_QS', 'DIPEP_QT', 'DIPEP_QV', 'DIPEP_QW',
'DIPEP_QY', 'DIPEP_RA', 'DIPEP_RC', 'DIPEP_RD', 'DIPEP_RE', 'DIPEP_RF',
'DIPEP_RG', 'DIPEP_RH', 'DIPEP_RI', 'DIPEP_RK', 'DIPEP_RL', 'DIPEP_RM',
'DIPEP_RN', 'DIPEP_RP', 'DIPEP_RQ', 'DIPEP_RR', 'DIPEP_RS', 'DIPEP_RT',
'DIPEP_RV', 'DIPEP_RW', 'DIPEP_RY', 'DIPEP_SA', 'DIPEP_SC', 'DIPEP_SD',
'DIPEP_SE', 'DIPEP_SF', 'DIPEP_SG', 'DIPEP_SH', 'DIPEP_SI', 'DIPEP_SK',
'DIPEP_SL', 'DIPEP_SM', 'DIPEP_SN', 'DIPEP_SP', 'DIPEP_SQ', 'DIPEP_SR',
'DIPEP_SS', 'DIPEP_ST', 'DIPEP_SV', 'DIPEP_SW', 'DIPEP_SY', 'DIPEP_TA',
'DIPEP_TC', 'DIPEP_TD', 'DIPEP_TE', 'DIPEP_TF', 'DIPEP_TG', 'DIPEP_TH',
'DIPEP_TI', 'DIPEP_TK', 'DIPEP_TL', 'DIPEP_TM', 'DIPEP_TN', 'DIPEP_TP',

```
'DIPEP_TQ', 'DIPEP_TR', 'DIPEP_TS', 'DIPEP_TT', 'DIPEP_TV', 'DIPEP_TW',
'DIPEP_TY', 'DIPEP_VA', 'DIPEP_VC', 'DIPEP_VD', 'DIPEP_VE', 'DIPEP_VF',
'DIPEP_VG', 'DIPEP_VH', 'DIPEP_VI', 'DIPEP_VK', 'DIPEP_VL', 'DIPEP_VM',
'DIPEP_VN', 'DIPEP_VP', 'DIPEP_VQ', 'DIPEP_VR', 'DIPEP_VS', 'DIPEP_VT',
'DIPEP_VV', 'DIPEP_VW', 'DIPEP_VY', 'DIPEP_WA', 'DIPEP_WC', 'DIPEP_WD',
'DIPEP_WE', 'DIPEP_WF', 'DIPEP_WG', 'DIPEP_WH', 'DIPEP_WI', 'DIPEP_WK',
'DIPEP_WL', 'DIPEP_WM', 'DIPEP_WN', 'DIPEP_WP', 'DIPEP_WQ', 'DIPEP_WR',
'DIPEP_WS', 'DIPEP_WT', 'DIPEP_WV', 'DIPEP_WW', 'DIPEP_WY', 'DIPEP_YA',
'DIPEP_YC', 'DIPEP_YD', 'DIPEP_YE', 'DIPEP_YF', 'DIPEP_YG', 'DIPEP_YH',
'DIPEP_YI', 'DIPEP_YK', 'DIPEP_YL', 'DIPEP_YM', 'DIPEP_YN', 'DIPEP_YP',
'DIPEP_YQ', 'DIPEP_YR', 'DIPEP_YS', 'DIPEP_YT', 'DIPEP_YV', 'DIPEP_YW',
'DIPEP_YY', 'P_LOOP', 'KINASE2', 'GLPL', 'MHD', 'RNBS_A', 'RNBS_B', 'RNBS_C',
'RNBS_D', 'HRD', 'DFG', 'LRR1', 'LRR2', 'NUM_NLR_MOTIFS',
'num_motifs_detectados', 'dist_ploop_kinase2', 'dist_kinase2_glpl',
'dist_glpl_mhd', 'dist_ploop_kinase2_norm', 'dist_kinase2_glpl_norm',
'dist_glpl_mhd_norm', 'has_ploop', 'has_kinase2', 'has_glpl', 'has_mhd']

 Clusters ausentes:
168

Dataset original pós CD-HIT: (8644, 448)

--- Executando Balanceamento de Classes ---
Contagem inicial -> Positivos (R-genes): 1700 | Negativos (Background): 6944
Subamostragem aplicada com sucesso nos Negativos.
Contagem final   -> Positivos: 1700 | Negativos: 1700
Dataset Final Pronto: (3400, 448)


Finalizado em: 10:37:46

Tempo decorrido: 0:00:03.627026


Fim tratamento dos dados…
```

[12]:
```
'''
k-Fold-Cross-Validation

'''

def validacao_cruzada(nome, model, X_train, y_train, groups_train):

    cv = StratifiedGroupKFold(
        n_splits=5, #10
        shuffle=True,
        random_state=42
```

```
    )
    mcc_scorer = make_scorer(matthews_corrcoef)
    scoring_dict = {
        'recall': 'recall',
        'roc_auc': 'roc_auc',
        'mcc': mcc_scorer,
        'accuracy': 'accuracy',
        'precision': 'precision',
        'f1': 'f1'
    }
    scores = cross_validate(
        model,
        X_train,
        y_train,
        groups=groups_train,
        cv=cv,
        scoring=scoring_dict,
        n_jobs=-1
    )
    print(f"Finalizada a validação cruzada para: {nome}")
    return scores
```

[13]:
```
'''
Criação dos modelos/ Treinamento
'''
# para melhor separar os conjuntos de treino e teste considerando os␣
 ↪agrupamentos feitos no CD-HIT para sequências muito parecidas
def split_por_homologia(df):
    print(df.head(30))
    groups = df["cluster"]
    splitter = GroupShuffleSplit(
        n_splits=1,
        test_size=0.2,
        random_state=42
    )
    # cv  = StratifiedGroupKFold(
    #     n_splits = 10,
    #     shuffle=True,
    #     random_state = 42
    # )

    train_idx, test_idx = next(
        splitter.split(df, groups=groups)
        # cv.split(df, df["target"], groups)
    )
    train_df = df.iloc[train_idx]
```

```
    test_df = df.iloc[test_idx]
    return train_df, test_df

def calcular_e_salvar_shap(nome, model, X_test, feature_names, output_dir):
    """
    Calcula os SHAP values para modelos de árvore e salva os gráficos␣
 ↪explicativos.
    """
    print(f"\n[SHAP] Inicializando análise de interpretabilidade para: {nome}...
 ↪")

    # 1. Inicializa o explicador otimizado para árvores
    explainer = shap.TreeExplainer(model)

    # 2. Calcula os SHAP values para o conjunto de teste
    shap_values = explainer.shap_values(X_test)

    # 3. Tratamento de formato (Scikit-Learn Random Forest vs XGBoost)
    # O Random Forest do sklearn retorna uma lista contendo [valores_classe_0,␣
 ↪valores_classe_1]
    # O XGBoost retorna diretamente a matriz de SHAP values para a classe alvo
    if isinstance(shap_values, list):
        # Selecionamos o índice 1 (classe positiva: proteínas de resistência/
 ↪alvos)
        shap_values_pos = shap_values[1]
    elif isinstance(shap_values, np.ndarray) and len(shap_values.shape) == 3:
        # Tratamento para versões específicas do SHAP que geram arrays 3D
        shap_values_pos = shap_values[:, :, 1]
    else:
        shap_values_pos = shap_values

    # 4. Gráfico 1: Summary Plot (Beeswarm)
    # Mostra o impacto biológico de cada feature (ex: se alta frequência␣
 ↪aumenta ou diminui a predição)
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        max_display=20,  # Foco nas top 20 características
        show=False
    )
    # plt.title(f"SHAP Summary Plot (Top 20) - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_summary.png", bbox_inches="tight",␣
 ↪dpi=300)
```

```
    plt.close()

    # 5. Gráfico 2: Bar Plot (Importância Global)
    # Mostra a magnitude média do impacto de cada feature de forma absoluta
    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values_pos,
        X_test,
        feature_names=feature_names,
        plot_type="bar",
        max_display=20,
        show=False
    )
    # plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_bar.png", bbox_inches="tight",␣
↪dpi=300)
    plt.close()

    print(f"[SHAP] Gráficos salvos com sucesso em '{output_dir}/'!")


def calcular_shap_svm(nome, model, feature_names,  X_train, X_test, y_test,␣
↪output_dir):
    # ========================================================
    # IMPORTÂNCIA POR PERMUTAÇÃO PARA O SVM (Novo)
    # ========================================================
    print("\n[SVM] Calculando a importância por permutação dos atributos...")

    # Calcula a permutação no X_test (pode usar X_train se quiser focar no␣
↪ajuste ao treino)
    result_svm = permutation_importance(model, X_test, y_test, n_repeats=5,␣
↪random_state=42, n_jobs=-1)

    # Monta o DataFrame com os resultados
    importancia_svm = pd.DataFrame({
        'feature': columns,
        'importance_mean': result_svm.importances_mean
    }).sort_values('importance_mean', ascending=False)

    print("\nTop 20 features mais importantes para o SVM:")
    print(importancia_svm.head(20))

    # Gera o gráfico de barras igual ao do XGBoost
    top_svm = importancia_svm.head(20)
    plt.figure(figsize=(10,8))
```

```
    plt.barh(top_svm["feature"][::-1], top_svm["importance_mean"][::-1],␣
↪color='seagreen')
    plt.xlabel("Queda no Desempenho (Média)")
    plt.ylabel("Feature / Atributo")
    # plt.title("Top 20 Features - SVM Permutation Importance")
    plt.tight_layout()
    plt.savefig(f"{output_dir}/svm_importancia_permutacao.png",␣
↪bbox_inches="tight", dpi=300)
    plt.close()

    idx_back = np.random.choice(
        len(X_train),
        min(100,len(X_train)),
        replace = False
    )
    back = X_train[idx_back]

    idx_test = np.random.choice(
        len(X_test),
        min(200,len(X_test)),
        replace = False
    )
    x_test_sample = X_test[idx_test]

    print('\nCriando explainer shap...\n')
    explainer = shap.Explainer(model.predict, back)
    shap_values = explainer(x_test_sample, max_evals=1000)
    print("\nShap SHAP:",shap_values.values.shape)

    plt.figure(figsize=(12, 8))
    shap.summary_plot(
        shap_values.values,
        x_test_sample,
        feature_names=feature_names,
      #  plot_type="bar",
        max_display=20,
        show=False
    )
    # plt.title(f"SHAP Global Feature Importance - {nome}", fontsize=14, pad=20)
    plt.tight_layout()
    plt.savefig(f"{output_dir}/{nome}_shap_bar_summary.png",␣
↪bbox_inches="tight", dpi=300)
    plt.close()

    print(f"[SVM] Gráfico salvo com sucesso em '{output_dir}/
↪svm_importancia_permutacao.png'!")
```

```python
# ============================================================
# TREINAMENTO - treina os modelos
# ============================================================

def avaliar_modelo(nome, model, X_train, y_train, X_test, y_test):
    print("\n================================================================")
    print(nome)
    print("================================================================")
    model.fit(X_train, y_train)

    probs = model.predict_proba(X_test)[:,1]
    preds = (probs >= 0.5).astype(int)
    print(classification_report(y_test, preds))
    roc = roc_auc_score(y_test, probs)
    pr = average_precision_score(y_test, probs)
    mcc = matthews_corrcoef(y_test, preds)
    f1 = f1_score(y_test, preds)
    precision = precision_score(y_test, preds)
    recall = recall_score(y_test, preds)
    bal_acc = balanced_accuracy_score(y_test, preds)
    acc = accuracy_score(y_test, preds)

    print("ROC-AUC          :", roc)
    print("PR-AUC           :", pr)
    print("MCC              :", mcc)
    print("F1               :", f1)
    print("Precision        :", precision)
    print("Recall           :", recall)
    print("Balanced Accuracy:", bal_acc)
    print("Standard Accuracy:", acc)

    # ============================================================
    # MATRIZ DE CONFUSÃO
    # ============================================================
    cm = confusion_matrix(y_test, preds)
    plt.figure(figsize=(5,5))
    disp = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["Negativo", "Positivo"]
    )
    disp.plot(cmap="Blues")
    # plt.title(f"Matriz de Confusão - {nome}")
    plt.savefig(
        f"{OUTPUT_DIR}/{nome}_matriz_confusao.png",
        bbox_inches="tight"
    )
    plt.close()
```

```
    # ======================================================
    # CURVA ROC
    # ======================================================
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)
    plt.figure(figsize=(6,6))
    plt.plot(
        fpr,
        tpr,
        label=f"AUC = {roc_auc:.4f}"
    )
    plt.plot([0,1], [0,1], linestyle="--")
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    # plt.title(f"Curva ROC - {nome}")
    plt.legend(loc="lower right")
    plt.savefig(
        f"{OUTPUT_DIR}/{nome}_roc.png",
        bbox_inches="tight"
    )
    plt.close()

    return {
        "Modelo": nome,
        "ROC_AUC": roc,
        "F1": f1,
        "Precision": precision,
        "Recall": recall,
        "Balanced_Accuracy": bal_acc,
        "Accuracy": acc,
        "MCC":mcc
    }, model


# ======================================================
# Separação Treino/Teste 80/20
# SPLIT POR HOMOLOGIA
# ======================================================
inicio_treinamento = datetime.now()
print("\n\n\nInciando Treinamento dos modelos: ", inicio_treinamento.
  ↪strftime("%H:%M:%S"))

train_df, test_df = split_por_homologia(df)
X_train = train_df[columns]
y_train = train_df["target"]
X_test = test_df[columns]
y_test = test_df["target"]
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ======================================================
# MODELOS
# ======================================================

rf = RandomForestClassifier(
    n_estimators=1000,
    # max_depth = 12,
    max_depth = None,
    min_samples_split=5,
    max_features=0.2,
    random_state=42,
    class_weight="balanced",
    n_jobs = 1
)

xgb = XGBClassifier(
    n_estimators=1200,
    max_depth=7,
    learning_rate=0.05,
    # early_stopping_rounds=50,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss",
    random_state=42,
    min_child_weight=3,
    reg_alpha=0.5,
    reg_lambda=2,
    scale_pos_weight=1.0
)


svm = SVC(
    probability=True,
    kernel="rbf",
    class_weight="balanced",
    random_state=42,
    C=2,
    gamma='scale'
)
scores = []
groups_train = train_df["cluster"]

# msc_scorer = make_scorer(matthews_corrcoef)
```

```
scoring = {
    'mcc': make_scorer(matthews_corrcoef),
    'recall':  make_scorer(recall_score),
    'precision':  make_scorer(precision_score),
    'f1':  make_scorer(f1_score),
    'roc_auc': 'roc_auc',
}
```

```
Inciando Treinamento dos modelos:  10:37:46
    protein_length     AAC_A     AAC_C     AAC_D     AAC_E     AAC_F  \
0              231  0.090909  0.012987  0.069264  0.077922  0.051948
1              704  0.049716  0.007102  0.080966  0.117898  0.041193
2              971  0.045314  0.017508  0.054583  0.071061  0.036045
3              323  0.046440  0.027864  0.052632  0.080495  0.046440
4              929  0.034446  0.034446  0.046286  0.069968  0.053821
5              709  0.053597  0.014104  0.049365  0.071932  0.043724
6              999  0.053053  0.018018  0.070070  0.050050  0.036036
7             1071  0.056956  0.020542  0.059757  0.038282  0.043884
8             1050  0.043810  0.022857  0.053333  0.080952  0.041905
9              904  0.042035  0.022124  0.032080  0.055310  0.056416
10            1152  0.065972  0.015625  0.041667  0.046875  0.030382
11             949  0.033720  0.034773  0.052687  0.067439  0.041096
12             272  0.055147  0.033088  0.040441  0.084559  0.040441
13             428  0.049065  0.023364  0.070093  0.070093  0.051402
14             253  0.059289  0.019763  0.071146  0.051383  0.039526
15             886  0.038375  0.024831  0.060948  0.055305  0.050790
16             910  0.052747  0.017582  0.060440  0.065934  0.052747
17             879  0.035267  0.018203  0.052332  0.073948  0.046644
18             239  0.041841  0.020921  0.083682  0.050209  0.046025
19             126  0.055556  0.031746  0.055556  0.055556  0.039683
20             104  0.028846  0.038462  0.038462  0.067308  0.028846
21             105  0.057143  0.019048  0.076190  0.019048  0.057143
22             716  0.044693  0.023743  0.061453  0.067039  0.046089
23             196  0.061224  0.010204  0.040816  0.091837  0.035714
24             122  0.040984  0.008197  0.081967  0.073770  0.040984
25            1289  0.044220  0.017843  0.061288  0.074476  0.046548
26             846  0.034279  0.017730  0.063830  0.069740  0.047281
27             883  0.055493  0.019253  0.057758  0.070215  0.048698
28             906  0.052980  0.020971  0.055188  0.073951  0.049669
29             343  0.084548  0.011662  0.061224  0.064140  0.037901

       AAC_G     AAC_H     AAC_I     AAC_K  …  dist_ploop_kinase2_norm  \
0   0.077922  0.038961  0.043290  0.082251  …                -1.000000
1   0.048295  0.009943  0.065341  0.117898  …                -1.000000
```

2	0.055613	0.024717	0.057673	0.061792	…	0.094748
3	0.065015	0.024768	0.061920	0.052632	…	-1.000000
4	0.051668	0.027987	0.071044	0.059203	…	-1.000000
5	0.063470	0.021157	0.063470	0.077574	…	-1.000000
6	0.069069	0.021021	0.043043	0.077077	…	-1.000000
7	0.090570	0.023343	0.040149	0.059757	…	-1.000000
8	0.048571	0.024762	0.065714	0.058095	…	-0.023810
9	0.059735	0.028761	0.065265	0.074115	…	-1.000000
10	0.110243	0.026910	0.039062	0.046007	…	-1.000000
11	0.049526	0.025290	0.060063	0.052687	…	0.089568
12	0.044118	0.029412	0.047794	0.066176	…	-1.000000
13	0.053738	0.011682	0.072430	0.077103	…	-1.000000
14	0.063241	0.027668	0.075099	0.102767	…	0.280632
15	0.053047	0.027088	0.067720	0.082393	…	0.106095
16	0.040659	0.021978	0.071429	0.086813	…	0.100000
17	0.039818	0.036405	0.069397	0.059158	…	0.102389
18	0.033473	0.025105	0.092050	0.079498	…	-1.000000
19	0.055556	0.039683	0.079365	0.095238	…	-1.000000
20	0.067308	0.019231	0.067308	0.076923	…	-1.000000
21	0.057143	0.019048	0.066667	0.076190	…	-1.000000
22	0.051676	0.018156	0.064246	0.071229	…	-1.000000
23	0.035714	0.020408	0.071429	0.076531	…	-1.000000
24	0.032787	0.016393	0.049180	0.073770	…	-1.000000
25	0.038014	0.038014	0.071373	0.074476	…	0.070597
26	0.033097	0.027187	0.075650	0.065012	…	0.106383
27	0.049830	0.022650	0.056625	0.092865	…	0.098528
28	0.045254	0.029801	0.070640	0.071744	…	0.097130
29	0.075802	0.032070	0.075802	0.067055	…	-1.000000

	dist_kinase2_glpl_norm	dist_glpl_mhd_norm	has_ploop	has_kinase2	\
0	-1.000000	1	0	0	
1	-1.000000	0	0	1	
2	0.108136	1	1	1	
3	-1.000000	0	0	0	
4	-1.000000	1	0	1	
5	0.128350	0	0	1	
6	0.288288	1	0	1	
7	0.202614	0	0	1	
8	0.096190	1	1	1	
9	-1.000000	0	0	1	
10	-1.000000	0	0	1	
11	0.295047	1	1	1	
12	0.312500	0	0	1	
13	0.235981	1	0	1	
14	-1.000000	1	1	1	
15	0.093679	1	1	1	
16	-0.346154	1	1	1	
17	0.145620	1	1	1	

18	-1.000000	1	0	0
19	-1.000000	0	0	0
20	-1.000000	0	0	1
21	-1.000000	1	0	0
22	0.139665	1	0	1
23	-1.000000	0	1	0
24	-1.000000	0	0	0
25	-0.452289	1	1	1
26	0.102837	1	1	1
27	0.412231	1	1	1
28	0.143488	1	1	1
29	-1.000000	0	0	0

	has_glpl	has_mhd	target	id	cluster
0	0	1	1	NP_001234459.3	POS_1502
1	0	1	1	NP_001308492.1	POS_1005
2	1	4	1	CAQ5402917.1	POS_548
3	0	0	1	CAQ5403257.1	POS_1404
4	0	2	1	CAQ5404663.1	POS_614
5	1	2	1	CAQ5406153.1	POS_1000
6	1	3	1	CAQ5404100.1	POS_504
7	1	2	1	CAQ5404098.1	POS_427
8	1	4	1	CAQ5403806.1	POS_449
9	0	1	1	CAQ5409432.1	POS_681
10	0	1	1	CAQ5408022.1	POS_356
11	1	4	1	CAQ5407207.1	POS_585
12	1	2	1	CAQ5411306.1	POS_1456
13	1	3	1	CAQ5411102.1	POS_1278
14	0	3	1	CAQ5410995.1	POS_1472
15	1	4	1	CAQ5413609.1	POS_723
16	1	4	1	CAQ5413612.1	POS_663
17	1	4	1	CAQ5410853.1	POS_740
18	0	1	1	CAQ5410852.1	POS_1491
19	0	0	1	CAQ5413789.1	POS_1696
20	0	1	1	CAQ5410659.1	POS_1709
21	0	1	1	CAQ5410656.1	POS_1708
22	1	3	1	CAQ5410654.1	POS_994
23	1	2	1	CAQ5410652.1	POS_1573
24	1	1	1	CAQ5410649.1	POS_1699
25	1	4	1	CAQ5410600.1	POS_211
26	1	4	1	CAQ5413906.1	POS_810
27	1	4	1	CAQ5410544.1	POS_729
28	1	4	1	CAQ5413907.1	POS_675
29	0	0	1	CAQ5413931.1	POS_1378

[30 rows x 448 columns]

[14]:
```
inicio_otimizacao = datetime.now()
print("\n\n\nInciando Otimização dos modelos: ", inicio_otimizacao.strftime("%H:
  ↪%M:%S"))

cv_t = GroupKFold(n_splits=5)

param_dist_xgb = {
    'n_estimators':[500,800,1200,1500],
    'max_depth':[3,4,5,6,7,10],
    'learning_rate':[0.01,0.03,0.05,0.1],
    'subsample':[0.7,0.8,0.9],
    'colsample_bytree':[0.7,0.8,0.9],
    'min_child_weight':[1,3,5],
    'reg_alpha':[0,0.1,0.5,0.7],
    'reg_lambda':[1,2,5]
}
# Otimização XGB
print('\n\nOtimizando XGBoost...')
search_xgb = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_dist_xgb,
    n_iter=50,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_xgb.fit(
    X_train,
    y_train,
    groups=groups_train
)
print("\nMelhores parâmetros para o XGB:")
print(search_xgb.best_params_)

print("\nMelhor Recall:")
print(search_xgb.best_score_)

best_params = search_xgb.best_params_
xgb = search_xgb.best_estimator_
```

```
Inciando Otimização dos modelos:  10:37:46
```

```
Otimizando XGBoost…
Fitting 5 folds for each of 50 candidates, totalling 250 fits

Melhores parâmetros para o XGB:
{'subsample': 0.8, 'reg_lambda': 1, 'reg_alpha': 0.5, 'n_estimators': 1500,
'min_child_weight': 1, 'max_depth': 3, 'learning_rate': 0.05,
'colsample_bytree': 0.7}

Melhor Recall:
0.8920809070696771
```

[15]:
```python
# otimização RF

param_dist_rf = {
    'n_estimators':[500,1000,1200,1500],
    'max_depth':[8,12,16, None],
    'min_samples_split':[2,4,5,8],
    'min_samples_leaf':[1,2,4],
    'max_features':['sqrt','log2',0,3],
    'class_weight':['balanced','balanced_subsample']
}

print('\n\nOtimizando RF...')
search_rf = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist_rf,
    n_iter=40,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_rf.fit(
    X_train,
    y_train,
    groups=groups_train
)
print("\nMelhores parâmetros para o RF:")
print(search_rf.best_params_)

print("\nMelhor Recall:")
print(search_rf.best_score_)
```

```
rf = search_rf.best_estimator_

rf.set_params(n_jobs=-1)
```

```
Otimizando RF…
Fitting 5 folds for each of 40 candidates, totalling 200 fits

Melhores parâmetros para o RF:
{'n_estimators': 1000, 'min_samples_split': 8, 'min_samples_leaf': 1,
'max_features': 'sqrt', 'max_depth': None, 'class_weight': 'balanced_subsample'}

Melhor Recall:
0.8609705821511874
```

[15]:
```
RandomForestClassifier(class_weight='balanced_subsample', min_samples_split=8,
                       n_estimators=1000, n_jobs=-1, random_state=42)
```

[16]:
```
print('\n\nOtimizando SVM...')
param_dist_svm = {
    'C':[0.5,1,2,5,10,20,50],
    'gamma':[
        'scale',
        0.01,
        0.005,
        0.001,
        0.0005
    ],
    'kernel':['rbf']
}
search_svm = RandomizedSearchCV(
    estimator=svm,
    param_distributions=param_dist_svm,
    n_iter=25,
    scoring=scoring,
    refit='f1',
    cv=cv_t,
    random_state=42,
    verbose=2,
    n_jobs=-1
)

search_svm.fit(
    X_train,
    y_train,
    groups=groups_train
```

```
)
print("\nMelhores parâmetros para o SVM:")
print(search_svm.best_params_)

print("\nMelhor Recall:")
print(search_svm.best_score_)

svm = search_svm.best_estimator_

fim = datetime.now()
print("\n\nFinalizado otimização dos modelos em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio_otimizacao)
# Fim otimização
```

```
Otimizando SVM…
Fitting 5 folds for each of 25 candidates, totalling 125 fits

Melhores parâmetros para o SVM:
{'kernel': 'rbf', 'gamma': 0.001, 'C': 2}

Melhor Recall:
0.8899564834649676


Finalizado otimização dos modelos em: 10:53:05

Tempo decorrido: 0:15:19.242567
```

[17]:
```
# xgb = search_xgb.best_estimator_

print("\n\n Inciando validação cruzada");
scores_rf = validacao_cruzada('RF',rf,X_train, y_train, groups_train)
scores_xgb = validacao_cruzada('XGB',xgb, X_train, y_train, groups_train)
scores_svm = validacao_cruzada('SVM',svm, X_train, y_train, groups_train)

print("Cross-Validation:RF")
for k,v in scores_rf.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")

print("Cross-Validation XGB")
for k,v in scores_xgb.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")
```

```
print("Cross-Validation SVM")
for k,v in scores_svm.items():
    if "test" in k:
        print(f"{k}: {v.mean():.4f} ± {v.std():.4f}")


resultados_cv = pd.DataFrame({
    'Modelo': ['RF','XGB','SVM'],
    'Accuracy': [
        scores_rf['test_accuracy'].mean(),
        scores_xgb['test_accuracy'].mean(),
        scores_svm['test_accuracy'].mean(),
    ],
    'Recall': [
        scores_rf['test_recall'].mean(),
        scores_xgb['test_recall'].mean(),
        scores_svm['test_recall'].mean(),
    ],
    'Precision': [
        scores_rf['test_precision'].mean(),
        scores_xgb['test_precision'].mean(),
        scores_svm['test_precision'].mean(),
    ],
    'F1': [
        scores_rf['test_f1'].mean(),
        scores_xgb['test_f1'].mean(),
        scores_svm['test_f1'].mean(),
    ],
    'ROC_AUC': [
        scores_rf['test_roc_auc'].mean(),
        scores_xgb['test_roc_auc'].mean(),
        scores_svm['test_roc_auc'].mean(),
    ],
    'MCC': [
        scores_rf['test_mcc'].mean(),
        scores_xgb['test_mcc'].mean(),
        scores_svm['test_mcc'].mean(),
    ]
})
print(resultados_cv)
resultados_cv.to_csv(
        f"{OUTPUT_DIR}/metricas_cv.csv",
        index=False
    )
dados = []

for score in scores_rf['test_roc_auc']:
```

```
    dados.append(['RF',score])
for score in scores_xgb['test_roc_auc']:
    dados.append(['XGB',score])
for score in scores_svm['test_roc_auc']:
    dados.append(['SVM',score])

grafico = pd.DataFrame(dados, columns=['Modelo','ROC_AUC'])
grafico.boxplot(by='Modelo')
plt.ylabel("ROC-AUC")
# plt.title("10-Fold Cross Validation")
plt.suptitle("")
plt.tight_layout()
# plt.show()
plt.savefig(
    f"{OUTPUT_DIR}/kfold_cross_validation_roc.png",
    bbox_inches="tight"
)
plt.close()
```

```
 Inciando validação cruzada
Finalizada a validação cruzada para: RF
Finalizada a validação cruzada para: XGB
Finalizada a validação cruzada para: SVM
Cross-Validation:RF
test_recall: 0.7692 ± 0.0139
test_roc_auc: 0.9397 ± 0.0082
test_mcc: 0.7708 ± 0.0146
test_accuracy: 0.8757 ± 0.0081
test_precision: 0.9815 ± 0.0052
test_f1: 0.8624 ± 0.0099
Cross-Validation XGB
test_recall: 0.8512 ± 0.0209
test_roc_auc: 0.9424 ± 0.0089
test_mcc: 0.8015 ± 0.0207
test_accuracy: 0.8985 ± 0.0107
test_precision: 0.9432 ± 0.0131
test_f1: 0.8946 ± 0.0119
Cross-Validation SVM
test_recall: 0.8440 ± 0.0205
test_roc_auc: 0.9484 ± 0.0085
test_mcc: 0.8070 ± 0.0204
test_accuracy: 0.9004 ± 0.0103
test_precision: 0.9546 ± 0.0189
test_f1: 0.8956 ± 0.0112
  Modelo  Accuracy    Recall  Precision        F1   ROC_AUC       MCC
0     RF  0.875735  0.769228   0.981456  0.862423  0.939706  0.770808
```

```
1    XGB  0.898529  0.851225   0.943151  0.894628  0.942398  0.801468
2    SVM  0.900368  0.843984   0.954608  0.895586  0.948439  0.806975
```

[18]:
```
################################################
# Treinamento
inicio = datetime.now()
print("\n\n\nInciando treinamento RF: ", inicio.strftime("%H:%M:%S"))
results = []
r1, rf_model = avaliar_modelo(
    "Random Forest",
    rf,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r1)
fim = datetime.now()
print("\n\nFinalizado treinamento RF em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("Random Forest", rf_model, X_test, columns,OUTPUT_DIR )

inicio = datetime.now()
print("\n\n\nInciando treinamento XGBoost: ", inicio.strftime("%H:%M:%S"))
r2, xgb_model = avaliar_modelo(
    "XGBoost",
    xgb,
    X_train,
    y_train,
    X_test,
    y_test
)
results.append(r2)
fim = datetime.now()
print("\n\nFinalizado em XGBoost:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_e_salvar_shap("XGBoost", xgb_model, X_test, columns,OUTPUT_DIR )

inicio = datetime.now()
print("\n\n\nInciando treinamento SVM: ", inicio.strftime("%H:%M:%S"))
r3, svm_model = avaliar_modelo(
    "SVM",
    svm,
    X_train,
    y_train,
```

```
    X_test,
    y_test
)
results.append(r3)
fim = datetime.now()
print("\n\nFinalizado em SVM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

calcular_shap_svm("SVM", svm_model,columns, X_train, X_test, y_test, OUTPUT_DIR)

fim = datetime.now()
print("\n\nFinalizado fase de treinamento em:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio_treinamento)

print("\nFim criação/treino dos modelos\n")
```

```
Inciando treinamento RF:  10:53:27

============================================================
Random Forest
============================================================
              precision    recall  f1-score   support

           0       0.85      0.98      0.91       358
           1       0.98      0.80      0.88       322

    accuracy                           0.90       680
   macro avg       0.91      0.89      0.90       680
weighted avg       0.91      0.90      0.90       680

ROC-AUC           : 0.9542749574933203
PR-AUC            : 0.9618177020542162
MCC               : 0.8037186718808346
F1                : 0.8805460750853242
Precision         : 0.9772727272727273
Recall            : 0.8012422360248447
Balanced Accuracy: 0.8922412297442659
Standard Accuracy: 0.8970588235294118


Finalizado treinamento RF em: 10:53:29

Tempo decorrido: 0:00:02.270760

[SHAP] Inicializando análise de interpretabilidade para: Random Forest…
```

```
[SHAP] Gráficos salvos com sucesso em 'pipelineAG/'!



Inciando treinamento XGBoost:  10:54:01

============================================================
XGBoost
============================================================
              precision    recall  f1-score   support

           0       0.90      0.96      0.92       358
           1       0.95      0.88      0.91       322

    accuracy                           0.92       680
   macro avg       0.92      0.92      0.92       680
weighted avg       0.92      0.92      0.92       680

ROC-AUC          : 0.963166660883445
PR-AUC           : 0.9644334860365208
MCC              : 0.8363236524095334
F1               : 0.9096774193548387
Precision        : 0.9463087248322147
Recall           : 0.8757763975155279
Balanced Accuracy: 0.9155418300426802
Standard Accuracy: 0.9176470588235294


Finalizado em XGBoost: 10:54:07

Tempo decorrido: 0:00:05.842443

[SHAP] Inicializando análise de interpretabilidade para: XGBoost…
[SHAP] Gráficos salvos com sucesso em 'pipelineAG/'!



Inciando treinamento SVM:  10:54:08

============================================================
SVM
============================================================
              precision    recall  f1-score   support

           0       0.90      0.94      0.92       358
           1       0.93      0.89      0.91       322

    accuracy                           0.92       680
```

```
   macro avg       0.92      0.91      0.92       680
weighted avg       0.92      0.92      0.92       680

ROC-AUC          : 0.9570335542524029
PR-AUC           : 0.9615403671046813
MCC              : 0.8324869972206768
F1               : 0.9090909090909091
Precision        : 0.9344262295081968
Recall           : 0.8850931677018633
Balanced Accuracy: 0.9146136229570769
Standard Accuracy: 0.9161764705882353


Finalizado em SVM: 10:54:09

Tempo decorrido: 0:00:01.574313

[SVM] Calculando a importância por permutação dos atributos…

Top 20 features mais importantes para o SVM:
      feature  importance_mean
110  DIPEP_FL         0.004412
326  DIPEP_SG         0.003824
38   DIPEP_AV         0.003529
143  DIPEP_HD         0.003529
204  DIPEP_LE         0.003529
187  DIPEP_KH         0.003529
346  DIPEP_TG         0.003235
36   DIPEP_AS         0.003235
175  DIPEP_IR         0.003235
209  DIPEP_LK         0.003235
270  DIPEP_PL         0.003235
124  DIPEP_GE         0.002941
144  DIPEP_HE         0.002941
215  DIPEP_LR         0.002941
203  DIPEP_LD         0.002941
9       AAC_K         0.002941
13      AAC_P         0.002941
315  DIPEP_RR         0.002941
126  DIPEP_GG         0.002941
155  DIPEP_HR         0.002941

Criando explainer shap…

PermutationExplainer explainer:   8%|          | 17/200 [04:43<53:38, 17.59s/it]

[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
```

```
subsample=0.9; total time=  20.1s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  43.6s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.2min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  52.0s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  37.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  33.9s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  35.3s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  47.7s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  28.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  48.7s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  25.5s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  37.4s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  35.5s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   4.9s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   4.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  12.6s
```

```
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   6.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   7.5s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  18.9s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  17.8s
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  31.3s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  19.9s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  24.7s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  24.9s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time=  37.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  19.9s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  42.0s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.2min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  35.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  34.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  35.6s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  43.2s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  26.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  38.4s
```

```
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  35.4s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  58.3s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  46.2s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  25.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   5.0s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  11.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  11.1s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=   8.2s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=   8.3s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=   9.8s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  11.1s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  19.4s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  19.8s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  16.2s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time=  36.8s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  22.0s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time=  40.4s
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  23.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  17.9s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  42.4s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.5min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.1min
```

```
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  15.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  46.8s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  29.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  20.6s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  37.7s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  26.3s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  14.2s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  40.7s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   4.1s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   2.7s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.2s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   7.3s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   7.3s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  20.8s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  21.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  23.7s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time=  38.6s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  19.1s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  18.4s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time=  36.2s
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time=  29.6s
```

```
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  15.1s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  29.7s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  39.8s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  41.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  55.0s
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  45.3s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.1min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  16.3s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  47.2s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  29.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  21.8s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.0min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  35.1s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  24.7s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  38.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  35.7s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   5.1s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
```

```
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  12.6s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   7.7s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   9.3s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   6.2s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time=  40.4s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  20.2s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  19.6s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time=  37.8s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time=  37.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  18.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  41.2s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.3min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  52.7s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  50.9s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  40.7s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.6min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  31.4s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
```

```
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  13.6s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  26.2s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  13.9s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  45.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   5.0s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  11.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  21.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   4.9s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   4.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   6.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   6.1s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  23.5s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time=  41.2s
[CV] END …C=5, gamma=0.01, kernel=rbf; total time=  40.0s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time=  37.0s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time=  36.6s
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=   8.1s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
```

```
subsample=0.7; total time=  29.6s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  41.0s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  43.5s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  52.8s
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  44.7s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.4min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  43.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  25.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  20.6s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  36.6s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  20.3s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  38.5s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  35.0s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=   8.1s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   5.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  12.6s
```

```
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  10.8s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   9.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   6.1s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time=  32.7s
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  11.6s
[CV] END …C=5, gamma=0.01, kernel=rbf; total time=  31.7s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time=  26.7s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time=  28.5s
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  14.7s
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  13.5s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  17.3s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  42.6s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.6min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.0min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.4min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  45.1s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  24.8s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  36.9s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  35.4s
```

```
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  56.3s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  45.5s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  26.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   5.0s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   5.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  11.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=500; total time=  11.2s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=   8.1s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=1000; total time=   8.2s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=   9.8s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   7.6s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   9.1s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=2, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   6.1s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time=  40.5s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  20.3s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  24.4s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time=  37.7s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time=  34.3s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time=  59.3s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.3min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
```

```
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.1min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  17.2s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  46.1s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  28.1s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  21.0s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  37.1s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  22.0s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  38.3s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  35.7s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   5.3s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=   9.9s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   6.3s
```

```
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  22.2s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  16.4s
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  23.2s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  20.6s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  19.9s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time=  36.6s
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time=  30.2s
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  14.7s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  30.3s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  40.2s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  42.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  53.2s
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  47.5s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.4min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  45.7s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  25.7s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  48.0s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.2min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  20.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  14.3s
```

```
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  39.6s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   4.1s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   2.6s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.3s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   7.3s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  23.5s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   8.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   6.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  10.8s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  23.2s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time=  34.5s
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  11.8s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  13.8s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  13.4s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  14.7s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  16.4s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time=  28.6s
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  14.9s
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  14.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  19.1s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
```

```
total time=  44.2s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.3min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  51.0s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  51.4s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  39.4s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.4min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  32.2s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  12.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.5min
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  24.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  20.5s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  21.1s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  22.6s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  20.0s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  15.1s
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  17.3s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  20.4s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  22.1s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time=  40.6s
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  24.3s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  17.9s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  42.3s
```

```
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.4min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  51.3s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  53.8s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  39.6s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  46.0s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  30.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  20.9s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=3,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=5,
subsample=0.9; total time=  35.7s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  19.5s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=5,
subsample=0.9; total time=  37.7s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  20.8s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   4.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   2.7s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.3s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  23.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  23.4s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  10.9s
```

```
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  19.0s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  14.9s
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  30.8s
[CV] END …C=5, gamma=0.01, kernel=rbf; total time=  39.9s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  24.6s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time=  36.8s
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  15.0s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  19.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  41.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  35.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  35.2s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  36.0s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  47.7s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  25.7s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  38.7s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  35.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  57.2s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  44.9s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
```

```
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  25.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   5.1s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  20.7s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  21.2s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=   9.9s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   6.2s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  24.3s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time=  32.9s
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  10.7s
[CV] END …C=5, gamma=0.01, kernel=rbf; total time=  31.4s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time=  27.3s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time=  29.2s
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  14.9s
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  11.9s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time=  58.5s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.2min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  36.9s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
subsample=0.8; total time=  35.1s
```

```
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  38.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  45.3s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  27.0s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  47.8s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  59.9s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.5min
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  23.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  20.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  20.3s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  22.6s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time=  38.0s
[CV] END …C=1, gamma=0.001, kernel=rbf; total time=  20.9s
[CV] END …C=50, gamma=0.001, kernel=rbf; total time=  20.3s
[CV] END …C=20, gamma=0.005, kernel=rbf; total time=  37.5s
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time=  30.2s
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  12.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time=  57.7s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  36.2s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.1, reg_lambda=2,
```

```
subsample=0.8; total time=  35.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  35.9s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=10,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  48.1s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  27.6s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  48.6s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  58.4s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.4min
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   4.1s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   2.6s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=None, max_features=0,
min_samples_leaf=1, min_samples_split=4, n_estimators=1200; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   7.3s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=2, n_estimators=1500; total time=   7.3s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  20.8s
[CV] END class_weight=balanced, max_depth=8, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=1200; total time=  21.1s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  23.0s
[CV] END …C=10, gamma=0.01, kernel=rbf; total time=  35.0s
```

```
[CV] END …C=5, gamma=0.0005, kernel=rbf; total time=  15.6s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  19.8s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  24.0s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time=  40.0s
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  25.3s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  20.6s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time= 1.1min
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=6,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  43.6s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time= 1.3min
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  52.4s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  50.6s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  40.9s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.5min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  31.6s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  13.8s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.5min
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=4, min_samples_split=2, n_estimators=500; total time=   5.2s
[CV] END class_weight=balanced, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=4, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  20.9s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  21.2s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   6.0s
```

```
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   6.2s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  19.4s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  20.8s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  15.3s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time=  36.3s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  22.1s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time=  40.1s
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  25.5s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time=  59.6s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0.1, reg_lambda=5,
subsample=0.8; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=7,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time= 2.2min
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=4,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  49.8s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.7;
total time=  41.0s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.6min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  30.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  13.7s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=4,
min_child_weight=1, n_estimators=500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  26.3s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  15.0s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  38.6s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   4.2s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
```

```
min_samples_leaf=1, min_samples_split=8, n_estimators=500; total time=   2.6s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=2, min_samples_split=2, n_estimators=1000; total time=   6.2s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=0,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  23.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1000; total time=  23.5s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=  10.9s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=  18.8s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  14.6s
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  30.9s
[CV] END …C=5, gamma=0.01, kernel=rbf; total time=  39.9s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  24.9s
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time=  30.4s
[CV] END …C=50, gamma=scale, kernel=rbf; total time=  20.1s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  29.8s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  40.1s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  42.8s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  54.2s
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  45.6s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.0min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.4min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
```

```
subsample=0.7; total time=  44.8s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  25.1s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  40.9s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  34.0s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  57.1s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  43.2s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=4,
min_child_weight=5, n_estimators=800, reg_alpha=0.7, reg_lambda=5,
subsample=0.8; total time=  26.2s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  24.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  20.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=sqrt,
min_samples_leaf=4, min_samples_split=5, n_estimators=1000; total time=  20.8s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  22.8s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  19.5s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  16.8s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time=  37.4s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time=  37.7s
[CV] END …C=0.5, gamma=0.005, kernel=rbf; total time=  30.7s
[CV] END …C=10, gamma=0.001, kernel=rbf; total time=  17.0s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=3,
min_child_weight=3, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  29.4s
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=500, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  39.4s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=3, n_estimators=1200, reg_alpha=0, reg_lambda=1, subsample=0.8;
total time=  43.8s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  53.2s
[CV] END colsample_bytree=0.8, learning_rate=0.03, max_depth=6,
```

```
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time=  46.3s
[CV] END colsample_bytree=0.9, learning_rate=0.05, max_depth=6,
min_child_weight=3, n_estimators=1500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.1min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=6,
min_child_weight=5, n_estimators=1200, reg_alpha=0.7, reg_lambda=1,
subsample=0.9; total time= 1.6min
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.7; total time=  45.9s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=6,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  27.3s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=5, subsample=0.8;
total time=  47.8s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=3, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=3,
min_child_weight=3, n_estimators=800, reg_alpha=0.1, reg_lambda=1,
subsample=0.7; total time=  20.9s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=3,
min_child_weight=5, n_estimators=500, reg_alpha=0.5, reg_lambda=1,
subsample=0.7; total time=  13.8s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=7,
min_child_weight=3, n_estimators=500, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time=  45.2s
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  23.5s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   9.2s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   9.2s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   5.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   5.1s
[CV] END class_weight=balanced, max_depth=8, max_features=log2,
min_samples_leaf=2, min_samples_split=4, n_estimators=1200; total time=  10.0s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1200; total time=   6.2s
[CV] END class_weight=balanced, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=  22.0s
[CV] END …C=10, gamma=0.0005, kernel=rbf; total time=  15.8s
```

```
[CV] END …C=5, gamma=scale, kernel=rbf; total time=  31.6s
[CV] END …C=1, gamma=0.0005, kernel=rbf; total time=  20.2s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  24.9s
[CV] END …C=1, gamma=scale, kernel=rbf; total time=  24.5s
[CV] END …C=50, gamma=0.005, kernel=rbf; total time=  37.9s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=4,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.9; total time= 1.0min
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.4min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.2min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  16.8s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=3,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=1,
subsample=0.8; total time=  45.1s
[CV] END colsample_bytree=0.8, learning_rate=0.05, max_depth=7,
min_child_weight=5, n_estimators=500, reg_alpha=0, reg_lambda=1, subsample=0.7;
total time=  28.5s
[CV] END colsample_bytree=0.7, learning_rate=0.05, max_depth=5,
min_child_weight=5, n_estimators=800, reg_alpha=0.5, reg_lambda=5,
subsample=0.7; total time=  38.2s
[CV] END colsample_bytree=0.7, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=500, reg_alpha=0.5, reg_lambda=5,
subsample=0.9; total time=  35.1s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=5,
min_child_weight=1, n_estimators=800, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  59.3s
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  47.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=sqrt,
min_samples_leaf=1, min_samples_split=4, n_estimators=1500; total time=  35.5s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   5.0s
[CV] END class_weight=balanced, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=2, n_estimators=500; total time=   5.0s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  12.7s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=3,
min_samples_leaf=2, min_samples_split=4, n_estimators=1000; total time=   6.3s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   7.8s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
```

```
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   9.0s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   6.3s
[CV] END …C=2, gamma=0.001, kernel=rbf; total time=  18.5s
[CV] END …C=20, gamma=0.0005, kernel=rbf; total time=  17.1s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time=  36.8s
[CV] END …C=0.5, gamma=0.0005, kernel=rbf; total time=  21.7s
[CV] END …C=2, gamma=0.01, kernel=rbf; total time=  40.1s
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  20.8s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=5, n_estimators=800, reg_alpha=0, reg_lambda=1, subsample=0.9;
total time=  18.4s
[CV] END colsample_bytree=0.9, learning_rate=0.1, max_depth=5,
min_child_weight=5, n_estimators=1500, reg_alpha=0.7, reg_lambda=2,
subsample=0.7; total time=  44.2s
[CV] END colsample_bytree=0.9, learning_rate=0.01, max_depth=6,
min_child_weight=1, n_estimators=1200, reg_alpha=0.1, reg_lambda=2,
subsample=0.9; total time= 2.4min
[CV] END colsample_bytree=0.8, learning_rate=0.01, max_depth=5,
min_child_weight=1, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.9; total time= 2.0min
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=3,
min_child_weight=1, n_estimators=500, reg_alpha=0, reg_lambda=2, subsample=0.9;
total time=  15.4s
[CV] END colsample_bytree=0.7, learning_rate=0.01, max_depth=10,
min_child_weight=3, n_estimators=1500, reg_alpha=0.5, reg_lambda=2,
subsample=0.7; total time= 2.5min
[CV] END colsample_bytree=0.8, learning_rate=0.1, max_depth=4,
min_child_weight=5, n_estimators=1200, reg_alpha=0, reg_lambda=5, subsample=0.7;
total time=  32.1s
[CV] END colsample_bytree=0.7, learning_rate=0.03, max_depth=3,
min_child_weight=3, n_estimators=500, reg_alpha=0.1, reg_lambda=1,
subsample=0.9; total time=  13.1s
[CV] END colsample_bytree=0.9, learning_rate=0.03, max_depth=10,
min_child_weight=1, n_estimators=800, reg_alpha=0.7, reg_lambda=1,
subsample=0.7; total time= 1.6min
[CV] END class_weight=balanced, max_depth=12, max_features=0,
min_samples_leaf=2, min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=sqrt,
min_samples_leaf=1, min_samples_split=8, n_estimators=1000; total time=  23.8s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   8.6s
[CV] END class_weight=balanced_subsample, max_depth=None, max_features=3,
min_samples_leaf=4, min_samples_split=5, n_estimators=1500; total time=   8.8s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
```

```
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=8, max_features=0, min_samples_leaf=2,
min_samples_split=8, n_estimators=500; total time=   0.0s
[CV] END class_weight=balanced, max_depth=12, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   4.9s
[CV] END class_weight=balanced_subsample, max_depth=12, max_features=log2,
min_samples_leaf=1, min_samples_split=8, n_estimators=1200; total time=  12.5s
[CV] END class_weight=balanced_subsample, max_depth=8, max_features=3,
min_samples_leaf=2, min_samples_split=8, n_estimators=1000; total time=   5.2s
[CV] END class_weight=balanced, max_depth=16, max_features=3,
min_samples_leaf=4, min_samples_split=8, n_estimators=1500; total time=   7.6s
[CV] END class_weight=balanced, max_depth=None, max_features=log2,
min_samples_leaf=4, min_samples_split=8, n_estimators=1000; total time=   9.2s
[CV] END class_weight=balanced_subsample, max_depth=16, max_features=3,
min_samples_leaf=1, min_samples_split=5, n_estimators=1000; total time=   6.2s
[CV] END …C=20, gamma=0.01, kernel=rbf; total time=  36.3s
[CV] END …C=2, gamma=0.005, kernel=rbf; total time=  26.1s
[CV] END …C=0.5, gamma=scale, kernel=rbf; total time=  16.1s
[CV] END …C=5, gamma=0.005, kernel=rbf; total time=  26.5s
[CV] END …C=0.5, gamma=0.01, kernel=rbf; total time=  28.5s
[CV] END …C=0.5, gamma=0.001, kernel=rbf; total time=  14.6s
[CV] END …C=50, gamma=0.01, kernel=rbf; total time=  14.2s

PermutationExplainer explainer: 201it [54:08, 16.24s/it]


Shap SHAP: (200, 445)
[SVM] Gráfico salvo com sucesso em 'pipelineAG/svm_importancia_permutacao.png'!


Finalizado fase de treinamento em: 11:48:49

Tempo decorrido: 1:11:03.465000

Fim criação/treino dos modelos


<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>

<Figure size 500x500 with 0 Axes>
```

[19]:
```
'''
Avaliação e Validação dos modelos
'''
print("\n\nInciando Validação dos modelos\n")
```

```
# verificar importância dos atributos/feature para o Random Forest
importancia = pd.DataFrame({"feature":columns,"importance":rf_model.
 ↪feature_importances_})
importancia = importancia.sort_values("importance",ascending = False)
print("\nTop 20 features: Random Forest")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh( top["feature"][::-1], top["importance"][::-1] )
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - Random Forest")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/rf_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

# verificar importância dos atributos/feature para o XGBoost
importancia = pd.DataFrame({"feature":columns, "importance":xgb_model.
 ↪feature_importances_})
importancia = importancia.sort_values("importance", ascending = False)
print("\nTop 20 features: XGBoost")
print(importancia.head(20))
# GRÁFICO
top = importancia.head(20)
plt.figure(figsize=(10,8))
plt.barh(top["feature"][::-1], top["importance"][::-1])
plt.xlabel("Importância")
plt.ylabel("Feature")
# plt.title("Top 20 Features - XGBoost")
plt.tight_layout()
plt.savefig(
    f"{OUTPUT_DIR}/xgb_importancia_variaveis.png",
    bbox_inches="tight"
)
plt.close()

print("\nCOMPARAÇÃO MODELOS")
print(pd.DataFrame(results))

results_df = pd.DataFrame(results)
results_df.to_csv(
        f"{OUTPUT_DIR}/metricas_comparacao.csv",
        index=False
```

```
    )

metrics = [
    "ROC_AUC",
    "F1",
    "Precision",
    "Recall",
    "Balanced_Accuracy"
]

for metric in metrics:
    plt.figure(figsize=(8,5))
    plt.bar(
        results_df["Modelo"],
        results_df[metric]
    )

    plt.ylim(0, 1)
    plt.ylabel(metric)
    # plt.title(f"Comparação dos Modelos - {metric}")
    for i, v in enumerate(results_df[metric]):
        plt.text(i, v + 0.01, f"{v:.3f}", ha='center')

    plt.savefig(
        f"{OUTPUT_DIR}/comparacao_{metric}.png",
        bbox_inches="tight"
    )
    plt.close()

modelos = {
    "Random Forest":rf_model,
    "XGBoost":xgb_model,
    "SVM":svm_model
    }
plt.figure(figsize=(7,7))
for nome, model in modelos.items():
    probs = model.predict_proba(X_test)[:,1]
    fpr, tpr, _ = roc_curve(y_test, probs)
    roc_auc = auc(fpr, tpr)
    plt.plot(
        fpr,
        tpr,
        label=f"{nome} AUC={roc_auc:.3f}"
    )
plt.plot([0,1],[0,1],'--')
plt.xlabel("FPR")
plt.ylabel("TPR")
```

```python
# plt.title("Comparação ROC")
plt.legend()
plt.savefig(f"{OUTPUT_DIR}/roc_comparativa.png")
plt.close()

print("Fim avaliação / validação modelos ")
```

```
Inciando Validação dos modelos


Top 20 features: Random Forest
                     feature  importance
10                     AAC_L    0.038210
444                  has_mhd    0.035311
433            NUM_NLR_MOTIFS    0.034540
440       dist_glpl_mhd_norm    0.024666
1                      AAC_A    0.024544
209                 DIPEP_LK    0.023640
422                 KINASE2    0.020788
441               has_ploop    0.019688
204                 DIPEP_LE    0.019556
421                   P_LOOP    0.018196
439   dist_kinase2_glpl_norm    0.018104
0              protein_length    0.016066
250                 DIPEP_NL    0.015859
13                     AAC_P    0.014667
438  dist_ploop_kinase2_norm    0.012703
190                 DIPEP_KL    0.012615
437            dist_glpl_mhd    0.012052
150                 DIPEP_HL    0.011909
215                 DIPEP_LR    0.010945
424                      MHD    0.010195

Top 20 features: XGBoost
                     feature  importance
444                  has_mhd    0.087298
440       dist_glpl_mhd_norm    0.043893
433            NUM_NLR_MOTIFS    0.041871
422                 KINASE2    0.034857
441               has_ploop    0.026817
10                     AAC_L    0.023234
1                      AAC_A    0.019438
438  dist_ploop_kinase2_norm    0.015903
421                   P_LOOP    0.012438
209                 DIPEP_LK    0.008515
13                     AAC_P    0.008453
```

```
0              protein_length    0.008417
143                 DIPEP_HD    0.007866
435       dist_ploop_kinase2    0.006555
150                 DIPEP_HL    0.006512
146                 DIPEP_HG    0.006019
382                 DIPEP_WC    0.005589
4                      AAC_E    0.005558
6                      AAC_G    0.005223
399                 DIPEP_WW    0.005206

COMPARAÇÃO MODELOS
          Modelo   ROC_AUC        F1  Precision    Recall  Balanced_Accuracy  \
0  Random Forest  0.954275  0.880546   0.977273  0.801242           0.892241
1        XGBoost  0.963167  0.909677   0.946309  0.875776           0.915542
2            SVM  0.957034  0.909091   0.934426  0.885093           0.914614

   Accuracy       MCC
0  0.897059  0.803719
1  0.917647  0.836324
2  0.916176  0.832487
Fim avaliação / validação modelos
```

[20]:

```
'''
Funções para rodar a aplicação
'''

df_validacao = []

def validar_motifs(df,planta,modelo):
    print("\nVALIDAÇÃO BIOLÓGICA\n")
    counts = {}
    for motif_name, pattern in motifs.items():
        counts[motif_name] = 0
        for seq in df["protein_seq"]:
            if re.search(pattern, seq):
                counts[motif_name] += 1
    for k, v in counts.items():
        print(k, ":", v)

    counts['planta'] = planta
    counts['modelo'] = modelo

    df_validacao.append(counts)
    df_counts = pd.DataFrame([counts])
    df_counts.to_csv(
        f"{OUTPUT_DIR}/validacao_biologica_{modelo}_{planta}.csv",
        index=False
```

```
    )



def rodar_transdecoder(fasta_nt):
    cmd1 = [
        "TransDecoder.LongOrfs",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output1.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd1,
                       stdout=log_file,
                       stderr=subprocess.STDOUT,
                       text=True)

    cmd2 = [
        "TransDecoder.Predict",
        "-t",
        fasta_nt,
        "--output_dir",
        f"{OUTPUT_DIR}"
    ]
    with open(fasta_nt+"_output2.txt", "a", encoding="utf-8") as log_file:
        subprocess.run(cmd2,
                       stdout=log_file,
                       stderr=subprocess.STDOUT,
                       text=True)
    pep_file = fasta_nt + ".transdecoder.pep"
    return pep_file


def predizer_transcritos(fasta_nt,model,nmodel, output, planta):
    probabilidade = 0.85
    candidatos = []
    pep_file = rodar_transdecoder(fasta_nt)
    for record in SeqIO.parse(fasta_nt+".transdecoder.pep", "fasta"):
        protein = str(record.seq)
        feats = extrair_features_proteina(protein)
        if feats is None:
            continue
        X = scaler.transform([feats])
        prob = model.predict_proba(X)[0][1]
        candidatos.append({
            "id": record.id,
```

```
            "descricao": record.description,
            "protein_length": len(protein),
            "probabilidade_resistencia": prob,
            "protein_seq": protein
        })

    candidatos_df = pd.DataFrame(candidatos)
    candidatos_df = candidatos_df.sort_values(
        "probabilidade_resistencia",
        ascending=False
    )

    # top = candidatos_df.head(40)
    top = candidatos_df[candidatos_df["probabilidade_resistencia"] >=␣
 ↪probabilidade]
    print("\nTOP CANDIDATOS com probabilidade de >= ")
    print(probabilidade)
    print(fasta_nt+", MODELO: "+nmodel)
    print(top[[
        "id",
        "descricao",
        "protein_length",
        "probabilidade_resistencia"
    ]])

    validar_motifs(top,planta,nmodel)

    candidatos_df.to_csv(
        f"{OUTPUT_DIR}/{output}",
        index=False
    )
```

[21]:
```
'''
Aplicação - Modelo XGBoost
'''
print("\n\n=================Inciando Aplicação: Predição de␣
 ↪Genes================\n\n")
print("\n\n Predição Modelo XGBoost")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))

print("\nPredição para ARABIDOPSIS\n")
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_arabidopsis_xgboost.
 ↪csv",planta="Arabidopsis thaliana")

fim = datetime.now()
```

```
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
================Inciando Aplicação: Predição de Genes===============
```

```
 Predição Modelo XGBoost
```

```
Inciando predição ARABIDOPSIS:  11:48:50

Predição para ARABIDOPSIS
```

```
TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/arabidopsis_transcriptoma.fasta, MODELO: XGBoost
                     id                                          descricao  \
4437  NM_001344576.1.p1  NM_001344576.1.p1 GENE.NM_001344576.1~~NM_0013…
4359  NM_001344486.1.p1  NM_001344486.1.p1 GENE.NM_001344486.1~~NM_0013…
4358  NM_001344485.1.p1  NM_001344485.1.p1 GENE.NM_001344485.1~~NM_0013…
4074  NM_001344104.1.p1  NM_001344104.1.p1 GENE.NM_001344104.1~~NM_0013…
8895     NM_122936.5.p1  NM_122936.5.p1 GENE.NM_122936.5~~NM_122936.5.p…
…                   …                                               …
5172  NM_001345509.1.p1  NM_001345509.1.p1 GENE.NM_001345509.1~~NM_0013…
5171  NM_001345508.1.p1  NM_001345508.1.p1 GENE.NM_001345508.1~~NM_0013…
5243  NM_001345607.1.p1  NM_001345607.1.p1 GENE.NM_001345607.1~~NM_0013…
5241  NM_001345605.1.p1  NM_001345605.1.p1 GENE.NM_001345605.1~~NM_0013…
1938  NM_001341343.1.p2  NM_001341343.1.p2 GENE.NM_001341343.1~~NM_0013…

      protein_length  probabilidade_resistencia
4437             907                   0.999980
4359             909                   0.999977
4358             909                   0.999977
4074             902                   0.999948
8895             902                   0.999948
…                …                          …
5172            1139                   0.850559
5171            1139                   0.850559
5243             580                   0.850503
5241             580                   0.850503
1938             506                   0.850056
```

```
[324 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 231
KINASE2 : 31
GLPL : 222
MHD : 219
RNBS_A : 0
RNBS_B : 243
RNBS_C : 297
RNBS_D : 303
HRD : 13
DFG : 66
LRR1 : 68
LRR2 : 130


Finalizado predição ARABIDOPSIS: 11:50:52

Tempo decorrido: 0:02:02.396194
```

[22]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_melongena_xgboost.csv",planta="Solanum␣
 ↪melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para MELONGENA




Inciando predição MELONGENA:  11:50:52

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/melongena_transcriptoma.fasta, MODELO: XGBoost
                     id                                                descricao  \
3221  GBEF01023314.1.p1  GBEF01023314.1.p1 GENE.GBEF01023314.1~~GBEF010…
3220  GBEF01023313.1.p1  GBEF01023313.1.p1 GENE.GBEF01023313.1~~GBEF010…
4489  GBEF01026077.1.p1  GBEF01026077.1.p1 GENE.GBEF01026077.1~~GBEF010…
4488  GBEF01026076.1.p1  GBEF01026076.1.p1 GENE.GBEF01026076.1~~GBEF010…
```

```
9238  GBEF01044458.1.p1  GBEF01044458.1.p1 GENE.GBEF01044458.1~~GBEF010…
…                  …                                                 …
5476  GBEF01028522.1.p1  GBEF01028522.1.p1 GENE.GBEF01028522.1~~GBEF010…
5488  GBEF01028531.1.p1  GBEF01028531.1.p1 GENE.GBEF01028531.1~~GBEF010…
5473  GBEF01028520.1.p1  GBEF01028520.1.p1 GENE.GBEF01028520.1~~GBEF010…
4458  GBEF01026020.1.p1  GBEF01026020.1.p1 GENE.GBEF01026020.1~~GBEF010…
3491  GBEF01023984.1.p1  GBEF01023984.1.p1 GENE.GBEF01023984.1~~GBEF010…

      protein_length  probabilidade_resistencia
3221             872                   0.999872
3220             881                   0.999754
4489             866                   0.999624
4488             892                   0.999622
9238             851                   0.999592
…                …                      …
5476             500                   0.857812
5488             500                   0.857812
5473             500                   0.857812
4458             937                   0.855964
3491             369                   0.851153

[105 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 46
KINASE2 : 14
GLPL : 60
MHD : 41
RNBS_A : 0
RNBS_B : 54
RNBS_C : 80
RNBS_D : 82
HRD : 3
DFG : 22
LRR1 : 7
LRR2 : 21


Finalizado predição MELONGENA: 11:52:38

Tempo decorrido: 0:01:46.032047
```

[23]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
```

```
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_tuberosum_xgboost.csv",planta="Solanum␣
 ↪tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para TUBEROSUM




Inciando predição TUBEROSUM:  11:52:38

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/tuberosum_transcriptoma.fasta, MODELO: XGBoost
                       id                                                 descricao  \
9242   XM_015312842.1.p1  XM_015312842.1.p1 GENE.XM_015312842.1~~XM_0153…
7736   XM_015304639.1.p1  XM_015304639.1.p1 GENE.XM_015304639.1~~XM_0153…
10090  XM_015314337.1.p1  XM_015314337.1.p1 GENE.XM_015314337.1~~XM_0153…
7408   XM_015304012.1.p1  XM_015304012.1.p1 GENE.XM_015304012.1~~XM_0153…
8916   XM_015312154.1.p1  XM_015312154.1.p1 GENE.XM_015312154.1~~XM_0153…
…                    …                                                  …
7494   XM_015304165.1.p1  XM_015304165.1.p1 GENE.XM_015304165.1~~XM_0153…
2441   XM_006357444.2.p1  XM_006357444.2.p1 GENE.XM_006357444.2~~XM_0063…
10091  XM_015314339.1.p2  XM_015314339.1.p2 GENE.XM_015314339.1~~XM_0153…
8323   XM_015310952.1.p1  XM_015310952.1.p1 GENE.XM_015310952.1~~XM_0153…
5792   XM_006365178.2.p1  XM_006365178.2.p1 GENE.XM_006365178.2~~XM_0063…

       protein_length  probabilidade_resistencia
9242             1216                   0.999997
7736             1023                   0.999994
10090             840                   0.999993
7408             3960                   0.999992
8916             1158                   0.999991
…                   …                          …
7494              869                   0.854148
2441              455                   0.853654
10091             187                   0.852993
8323              473                   0.850778
5792              608                   0.850176

[508 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA
```

```
P_LOOP : 335
KINASE2 : 180
GLPL : 376
MHD : 375
RNBS_A : 0
RNBS_B : 350
RNBS_C : 437
RNBS_D : 487
HRD : 19
DFG : 86
LRR1 : 100
LRR2 : 190


Finalizado predição TUBEROSUM: 11:54:44

Tempo decorrido: 0:02:05.400566
```

[24]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,␣
 ↪xgb_model,'XGBoost',output="predicoes_physalis_xgboost.csv",planta="Physalis␣
 ↪peruviana")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_xgboost.csv")
```

```
Predição para PHYSALIS




Inciando predição PHYSALIS:  11:54:44

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/physalis_transcriptoma.fasta, MODELO: XGBoost
                        id                                          descricao  \
10249  IABH01023929.1.p1  IABH01023929.1.p1 GENE.IABH01023929.1~~IABH010…
7797   IABH01019816.1.p1  IABH01019816.1.p1 GENE.IABH01019816.1~~IABH010…
9609   IABH01022911.1.p1  IABH01022911.1.p1 GENE.IABH01022911.1~~IABH010…
4194   IABH01013451.1.p1  IABH01013451.1.p1 GENE.IABH01013451.1~~IABH010…
9951   IABH01023453.1.p1  IABH01023453.1.p1 GENE.IABH01023453.1~~IABH010…
…                    …                                                 …
```

```
9815   IABH01023231.1.p1  IABH01023231.1.p1 GENE.IABH01023231.1~~IABH010…
6868   IABH01018357.1.p1  IABH01018357.1.p1 GENE.IABH01018357.1~~IABH010…
1415   IABH01005251.1.p1  IABH01005251.1.p1 GENE.IABH01005251.1~~IABH010…
10277  IABH01024037.1.p1  IABH01024037.1.p1 GENE.IABH01024037.1~~IABH010…
10356  IABH01024222.1.p1  IABH01024222.1.p1 GENE.IABH01024222.1~~IABH010…

       protein_length  probabilidade_resistencia
10249             530                   0.999989
7797              881                   0.999978
9609              879                   0.999964
4194              875                   0.999959
9951              387                   0.999959
…                 …                      …
9815             1477                   0.856804
6868              472                   0.856675
1415              290                   0.854205
10277            1017                   0.853518
10356             521                   0.851060

[187 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 94
KINASE2 : 31
GLPL : 114
MHD : 115
RNBS_A : 0
RNBS_B : 116
RNBS_C : 149
RNBS_D : 161
HRD : 7
DFG : 10
LRR1 : 11
LRR2 : 48


Finalizado predição TUBEROSUM: 11:56:46

Tempo decorrido: 0:02:02.110440

Arquivo salvo:
pipelineAG/predicoes_xgboost.csv
```

[25]:
```
print(df_validacao)
```

```
[{'P_LOOP': 231, 'KINASE2': 31, 'GLPL': 222, 'MHD': 219, 'RNBS_A': 0, 'RNBS_B':
243, 'RNBS_C': 297, 'RNBS_D': 303, 'HRD': 13, 'DFG': 66, 'LRR1': 68, 'LRR2':
```

```
130, 'planta': 'Arabidopsis thaliana', 'modelo': 'XGBoost'}, {'P_LOOP': 46,
'KINASE2': 14, 'GLPL': 60, 'MHD': 41, 'RNBS_A': 0, 'RNBS_B': 54, 'RNBS_C': 80,
'RNBS_D': 82, 'HRD': 3, 'DFG': 22, 'LRR1': 7, 'LRR2': 21, 'planta': 'Solanum
melongena', 'modelo': 'XGBoost'}, {'P_LOOP': 335, 'KINASE2': 180, 'GLPL': 376,
'MHD': 375, 'RNBS_A': 0, 'RNBS_B': 350, 'RNBS_C': 437, 'RNBS_D': 487, 'HRD': 19,
'DFG': 86, 'LRR1': 100, 'LRR2': 190, 'planta': 'Solanum tuberosum', 'modelo':
'XGBoost'}, {'P_LOOP': 94, 'KINASE2': 31, 'GLPL': 114, 'MHD': 115, 'RNBS_A': 0,
'RNBS_B': 116, 'RNBS_C': 149, 'RNBS_D': 161, 'HRD': 7, 'DFG': 10, 'LRR1': 11,
'LRR2': 48, 'planta': 'Physalis peruviana', 'modelo': 'XGBoost'}]
```

[26]:
```
'''
Aplicação - Modelo Random Forest
'''
print("\n\n Predição Modelo RF")
print("\nPredição para ARABIDOPSIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
 ↪rf_model,"RF",output="predicoes_arabidopsis_rf.csv",planta="Arabidopsis␣
 ↪thaliana")
fim = datetime.now()
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
 Predição Modelo RF

Predição para ARABIDOPSIS



Inciando predição ARABIDOPSIS:  11:56:46

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/arabidopsis_transcriptoma.fasta, MODELO: RF
                      id                                              descricao  \
9546     NM_124542.3.p1  NM_124542.3.p1 GENE.NM_124542.3~~NM_124542.3.p…
4720  NM_001344946.1.p1  NM_001344946.1.p1 GENE.NM_001344946.1~~NM_0013…
4723  NM_001344948.1.p1  NM_001344948.1.p1 GENE.NM_001344948.1~~NM_0013…
4375  NM_001344503.1.p1  NM_001344503.1.p1 GENE.NM_001344503.1~~NM_0013…
9182     NM_123739.3.p1  NM_123739.3.p1 GENE.NM_123739.3~~NM_123739.3.p…
…                   …                                                 …
1786  NM_001341144.1.p1  NM_001341144.1.p1 GENE.NM_001341144.1~~NM_0013…
9433     NM_124291.2.p1  NM_124291.2.p1 GENE.NM_124291.2~~NM_124291.2.p…
8490     NM_121841.2.p1  NM_121841.2.p1 GENE.NM_121841.2~~NM_121841.2.p…
```

```
8473     NM_121802.2.p1  NM_121802.2.p1 GENE.NM_121802.2~~NM_121802.2.p…
9026     NM_123373.3.p1  NM_123373.3.p1 GENE.NM_123373.3~~NM_123373.3.p…

      protein_length  probabilidade_resistencia
9546            1176                   0.983867
4720            1166                   0.983736
4723            1166                   0.983736
4375             849                   0.983613
9182             849                   0.983613
…                …                      …
1786             986                   0.859616
9433             981                   0.859494
8490             901                   0.858203
8473             781                   0.856147
9026            1018                   0.853654

[143 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 143
KINASE2 : 27
GLPL : 140
MHD : 126
RNBS_A : 0
RNBS_B : 134
RNBS_C : 142
RNBS_D : 143
HRD : 7
DFG : 38
LRR1 : 33
LRR2 : 86


Finalizado predição ARABIDOPSIS: 12:10:28

Tempo decorrido: 0:13:41.721205
```

[27]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
  ↪rf_model,'RF',output="predicoes_melongena_rf.csv",planta="Solanum melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para MELONGENA



Inciando predição MELONGENA:  12:10:28

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/melongena_transcriptoma.fasta, MODELO: RF
                     id                                                 descricao  \
3221  GBEF01023314.1.p1  GBEF01023314.1.p1 GENE.GBEF01023314.1~~GBEF010…
3220  GBEF01023313.1.p1  GBEF01023313.1.p1 GENE.GBEF01023313.1~~GBEF010…
9238  GBEF01044458.1.p1  GBEF01044458.1.p1 GENE.GBEF01044458.1~~GBEF010…
4489  GBEF01026077.1.p1  GBEF01026077.1.p1 GENE.GBEF01026077.1~~GBEF010…
4488  GBEF01026076.1.p1  GBEF01026076.1.p1 GENE.GBEF01026076.1~~GBEF010…
4337  GBEF01025817.1.p1  GBEF01025817.1.p1 GENE.GBEF01025817.1~~GBEF010…
4338  GBEF01025818.1.p1  GBEF01025818.1.p1 GENE.GBEF01025818.1~~GBEF010…
3222  GBEF01023315.1.p1  GBEF01023315.1.p1 GENE.GBEF01023315.1~~GBEF010…
3223  GBEF01023316.1.p1  GBEF01023316.1.p1 GENE.GBEF01023316.1~~GBEF010…
274   GBEF01001042.1.p1  GBEF01001042.1.p1 GENE.GBEF01001042.1~~GBEF010…
9236  GBEF01044457.1.p1  GBEF01044457.1.p1 GENE.GBEF01044457.1~~GBEF010…
789   GBEF01005143.1.p1  GBEF01005143.1.p1 GENE.GBEF01005143.1~~GBEF010…

      protein_length  probabilidade_resistencia
3221             872                   0.952496
3220             881                   0.949283
9238             851                   0.939927
4489             866                   0.930279
4488             892                   0.927160
4337             886                   0.926967
4338             886                   0.926967
3222             883                   0.914233
3223             874                   0.912567
274              867                   0.901904
9236             585                   0.873659
789              504                   0.857105

VALIDAÇÃO BIOLÓGICA

P_LOOP : 12
KINASE2 : 12
GLPL : 12
MHD : 7
RNBS_A : 0
RNBS_B : 3
RNBS_C : 8
RNBS_D : 12
```

```
HRD : 1
DFG : 0
LRR1 : 4
LRR2 : 7


Finalizado predição MELONGENA: 12:22:38

Tempo decorrido: 0:12:10.498027
```

[28]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
 ↪rf_model,"RF",output="predicoes_tuberosum_rf.csv",planta="Solanum tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para TUBEROSUM



Inciando predição TUBEROSUM:  12:22:38

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/tuberosum_transcriptoma.fasta, MODELO: RF
                       id                                              descricao  \
1960   XM_006356330.2.p1  XM_006356330.2.p1 GENE.XM_006356330.2~~XM_0063…
1986   XM_006356384.2.p1  XM_006356384.2.p1 GENE.XM_006356384.2~~XM_0063…
7736   XM_015304639.1.p1  XM_015304639.1.p1 GENE.XM_015304639.1~~XM_0153…
8918   XM_015312157.1.p1  XM_015312157.1.p1 GENE.XM_015312157.1~~XM_0153…
8919   XM_015312158.1.p1  XM_015312158.1.p1 GENE.XM_015312158.1~~XM_0153…
…                    …                                                 …
6124   XM_006366011.2.p1  XM_006366011.2.p1 GENE.XM_006366011.2~~XM_0063…
6125   XM_006366012.2.p1  XM_006366012.2.p1 GENE.XM_006366012.2~~XM_0063…
2869   XM_006358404.2.p1  XM_006358404.2.p1 GENE.XM_006358404.2~~XM_0063…
10092  XM_015314339.1.p1  XM_015314339.1.p1 GENE.XM_015314339.1~~XM_0153…
8189   XM_015305706.1.p1  XM_015305706.1.p1 GENE.XM_015305706.1~~XM_0153…

       protein_length  probabilidade_resistencia
1960             1279                   0.999847
1986             1646                   0.999484
7736             1023                   0.999130
8918             1258                   0.998991
```

```
8919              1258                    0.998991
…                 …                        …
6124               482                    0.863791
6125               481                    0.863533
2869              1239                    0.859994
10092              501                    0.854663
8189               506                    0.851823

[261 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 251
KINASE2 : 163
GLPL : 243
MHD : 243
RNBS_A : 0
RNBS_B : 217
RNBS_C : 246
RNBS_D : 260
HRD : 2
DFG : 52
LRR1 : 76
LRR2 : 124


Finalizado predição TUBEROSUM: 12:36:29

Tempo decorrido: 0:13:50.729747
```

[29]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,␣
  ↪rf_model,"RF",output="predicoes_physalis_rf.csv",planta="Physalis peruviana")
fim = datetime.now()
print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_rf.csv")
```

```
Predição para PHYSALIS
```

```
Inciando predição PHYSALIS:  12:36:29

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/physalis_transcriptoma.fasta, MODELO: RF
                      id                                                  descricao  \
10242  IABH01023913.1.p1  IABH01023913.1.p1 GENE.IABH01023913.1~~IABH010…
10241  IABH01023911.1.p1  IABH01023911.1.p1 GENE.IABH01023911.1~~IABH010…
10240  IABH01023909.1.p1  IABH01023909.1.p1 GENE.IABH01023909.1~~IABH010…
10938  IABH01029912.1.p1  IABH01029912.1.p1 GENE.IABH01029912.1~~IABH010…
711    IABH01002505.1.p1  IABH01002505.1.p1 GENE.IABH01002505.1~~IABH010…
710    IABH01002504.1.p1  IABH01002504.1.p1 GENE.IABH01002504.1~~IABH010…
4194   IABH01013451.1.p1  IABH01013451.1.p1 GENE.IABH01013451.1~~IABH010…
9609   IABH01022911.1.p1  IABH01022911.1.p1 GENE.IABH01022911.1~~IABH010…
10220  IABH01023861.1.p1  IABH01023861.1.p1 GENE.IABH01023861.1~~IABH010…
7797   IABH01019816.1.p1  IABH01019816.1.p1 GENE.IABH01019816.1~~IABH010…
9617   IABH01022919.1.p1  IABH01022919.1.p1 GENE.IABH01022919.1~~IABH010…
9265   IABH01022284.1.p1  IABH01022284.1.p1 GENE.IABH01022284.1~~IABH010…
10218  IABH01023856.1.p1  IABH01023856.1.p1 GENE.IABH01023856.1~~IABH010…
1483   IABH01005634.1.p1  IABH01005634.1.p1 GENE.IABH01005634.1~~IABH010…
9618   IABH01022921.1.p1  IABH01022921.1.p1 GENE.IABH01022921.1~~IABH010…
10219  IABH01023859.1.p1  IABH01023859.1.p1 GENE.IABH01023859.1~~IABH010…
10217  IABH01023853.1.p1  IABH01023853.1.p1 GENE.IABH01023853.1~~IABH010…
1820   IABH01007074.1.p1  IABH01007074.1.p1 GENE.IABH01007074.1~~IABH010…
4478   IABH01013954.1.p1  IABH01013954.1.p1 GENE.IABH01013954.1~~IABH010…
4477   IABH01013953.1.p1  IABH01013953.1.p1 GENE.IABH01013953.1~~IABH010…
10249  IABH01023929.1.p1  IABH01023929.1.p1 GENE.IABH01023929.1~~IABH010…
3828   IABH01012351.1.p1  IABH01012351.1.p1 GENE.IABH01012351.1~~IABH010…
39     IABH01000147.1.p1  IABH01000147.1.p1 GENE.IABH01000147.1~~IABH010…
10950  IABH01030004.1.p1  IABH01030004.1.p1 GENE.IABH01030004.1~~IABH010…
200    IABH01000745.1.p1  IABH01000745.1.p1 GENE.IABH01000745.1~~IABH010…
201    IABH01000746.1.p1  IABH01000746.1.p1 GENE.IABH01000746.1~~IABH010…
9483   IABH01022656.1.p1  IABH01022656.1.p1 GENE.IABH01022656.1~~IABH010…
9434   IABH01022586.1.p1  IABH01022586.1.p1 GENE.IABH01022586.1~~IABH010…
9480   IABH01022654.1.p1  IABH01022654.1.p1 GENE.IABH01022654.1~~IABH010…
725    IABH01002538.1.p1  IABH01002538.1.p1 GENE.IABH01002538.1~~IABH010…
724    IABH01002537.1.p1  IABH01002537.1.p1 GENE.IABH01002537.1~~IABH010…
5100   IABH01015140.1.p1  IABH01015140.1.p1 GENE.IABH01015140.1~~IABH010…
6495   IABH01017695.1.p1  IABH01017695.1.p1 GENE.IABH01017695.1~~IABH010…
2687   IABH01009585.1.p1  IABH01009585.1.p1 GENE.IABH01009585.1~~IABH010…
4958   IABH01014862.1.p1  IABH01014862.1.p1 GENE.IABH01014862.1~~IABH010…
4957   IABH01014861.1.p1  IABH01014861.1.p1 GENE.IABH01014861.1~~IABH010…
9067   IABH01021962.1.p1  IABH01021962.1.p1 GENE.IABH01021962.1~~IABH010…
9068   IABH01021963.1.p1  IABH01021963.1.p1 GENE.IABH01021963.1~~IABH010…
9069   IABH01021964.1.p1  IABH01021964.1.p1 GENE.IABH01021964.1~~IABH010…
9481   IABH01022655.1.p1  IABH01022655.1.p1 GENE.IABH01022655.1~~IABH010…
6822   IABH01018289.1.p1  IABH01018289.1.p1 GENE.IABH01018289.1~~IABH010…
7151   IABH01018818.1.p1  IABH01018818.1.p1 GENE.IABH01018818.1~~IABH010…
```

1171	IABH01004520.1.p1	IABH01004520.1.p1	GENE.IABH01004520.1~~IABH010…
1342	IABH01005037.1.p1	IABH01005037.1.p1	GENE.IABH01005037.1~~IABH010…
679	IABH01002433.1.p1	IABH01002433.1.p1	GENE.IABH01002433.1~~IABH010…
10261	IABH01024004.1.p1	IABH01024004.1.p1	GENE.IABH01024004.1~~IABH010…
10259	IABH01024000.1.p1	IABH01024000.1.p1	GENE.IABH01024000.1~~IABH010…
255	IABH01001092.1.p1	IABH01001092.1.p1	GENE.IABH01001092.1~~IABH010…
256	IABH01001093.1.p1	IABH01001093.1.p1	GENE.IABH01001093.1~~IABH010…
9950	IABH01023452.1.p1	IABH01023452.1.p1	GENE.IABH01023452.1~~IABH010…

	protein_length	probabilidade_resistencia
10242	852	0.993899
10241	852	0.993899
10240	849	0.993596
10938	890	0.993534
711	889	0.991638
710	889	0.991638
4194	875	0.990623
9609	879	0.983988
10220	865	0.981518
7797	881	0.980870
9617	881	0.979271
9265	960	0.978860
10218	866	0.973821
1483	908	0.972838
9618	873	0.971040
10219	870	0.970954
10217	726	0.964594
1820	892	0.963624
4478	864	0.961547
4477	864	0.961547
10249	530	0.960332
3828	952	0.958101
39	735	0.954580
10950	888	0.950587
200	891	0.945954
201	891	0.945954
9483	1390	0.941918
9434	903	0.934119
9480	1343	0.931953
725	1286	0.926557
724	1395	0.924382
5100	845	0.918804
6495	593	0.911007
2687	1247	0.904840
4958	884	0.895810
4957	884	0.895810
9067	962	0.891625
9068	962	0.891625

```
9069                962                    0.891625
9481                715                    0.887928
6822                762                    0.886373
7151               1208                    0.880647
1171               1326                    0.874061
1342                857                    0.873040
679                1174                    0.867375
10261               854                    0.865814
10259               854                    0.860882
255                 757                    0.859951
256                 757                    0.859951
9950                423                    0.855246

VALIDAÇÃO BIOLÓGICA

P_LOOP : 44
KINASE2 : 29
GLPL : 44
MHD : 49
RNBS_A : 0
RNBS_B : 47
RNBS_C : 49
RNBS_D : 50
HRD : 0
DFG : 5
LRR1 : 6
LRR2 : 26


Finalizado predição PHYSALIS: 12:51:17

Tempo decorrido: 0:14:48.408425

Arquivo salvo:
pipelineAG/predicoes_rf.csv
```

[30]:
```
'''
Aplicação - Modelo SVM
'''
print("\n\n Predição Modelo SVM")
print("\nPredição para ARABIDOPSIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição ARABIDOPSIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_arabidopsis_fasta,␣
 ↪svm_model,"SVM",output="predicoes_arabidopsis_svm.csv",planta="Arabidopsis␣
 ↪thaliana")
fim = datetime.now()
```

```
print("\n\nFinalizado predição ARABIDOPSIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
 Predição Modelo SVM

Predição para ARABIDOPSIS



Inciando predição ARABIDOPSIS:  12:51:17

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/arabidopsis_transcriptoma.fasta, MODELO: SVM
                     id                                                 descricao  \
9348     NM_124096.4.p1  NM_124096.4.p1 GENE.NM_124096.4~~NM_124096.4.p…
3611  NM_001343517.1.p1  NM_001343517.1.p1 GENE.NM_001343517.1~~NM_0013…
4374  NM_001344502.1.p1  NM_001344502.1.p1 GENE.NM_001344502.1~~NM_0013…
4376  NM_001344504.1.p1  NM_001344504.1.p1 GENE.NM_001344504.1~~NM_0013…
4373  NM_001344501.1.p1  NM_001344501.1.p1 GENE.NM_001344501.1~~NM_0013…
…                  …                                                  …
4378  NM_001344506.1.p1  NM_001344506.1.p1 GENE.NM_001344506.1~~NM_0013…
4377  NM_001344505.1.p1  NM_001344505.1.p1 GENE.NM_001344505.1~~NM_0013…
4405  NM_001344527.1.p1  NM_001344527.1.p1 GENE.NM_001344527.1~~NM_0013…
4511  NM_001344665.1.p1  NM_001344665.1.p1 GENE.NM_001344665.1~~NM_0013…
8648     NM_122172.3.p1  NM_122172.3.p1 GENE.NM_122172.3~~NM_122172.3.p…

      protein_length  probabilidade_resistencia
9348             844                   0.999984
3611            1296                   0.997007
4374             849                   0.996609
4376             849                   0.996609
4373             849                   0.996609
…                …                          …
4378             989                   0.853454
4377             989                   0.853454
4405             398                   0.852718
4511            1129                   0.852134
8648             316                   0.851111

[224 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 170
```

```
KINASE2 : 27
GLPL : 167
MHD : 148
RNBS_A : 0
RNBS_B : 183
RNBS_C : 211
RNBS_D : 214
HRD : 7
DFG : 43
LRR1 : 39
LRR2 : 99


Finalizado predição ARABIDOPSIS: 12:51:22

Tempo decorrido: 0:00:04.917625
```

[31]:
```
print("\nPredição para MELONGENA\n")
inicio = datetime.now()
print("\n\n\nInciando predição MELONGENA: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_melongena_fasta,␣
 ↪svm_model,'SVM',output="predicoes_melongena_svm.csv",planta="Solanum␣
 ↪melongena")
fim = datetime.now()
print("\n\nFinalizado predição MELONGENA:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para MELONGENA



Inciando predição MELONGENA:  12:51:22

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/melongena_transcriptoma.fasta, MODELO: SVM
                     id                                                descricao  \
4488  GBEF01026076.1.p1  GBEF01026076.1.p1 GENE.GBEF01026076.1~~GBEF010…
3221  GBEF01023314.1.p1  GBEF01023314.1.p1 GENE.GBEF01023314.1~~GBEF010…
4489  GBEF01026077.1.p1  GBEF01026077.1.p1 GENE.GBEF01026077.1~~GBEF010…
3223  GBEF01023316.1.p1  GBEF01023316.1.p1 GENE.GBEF01023316.1~~GBEF010…
789   GBEF01005143.1.p1  GBEF01005143.1.p1 GENE.GBEF01005143.1~~GBEF010…
3220  GBEF01023313.1.p1  GBEF01023313.1.p1 GENE.GBEF01023313.1~~GBEF010…
9160  GBEF01044249.1.p1  GBEF01044249.1.p1 GENE.GBEF01044249.1~~GBEF010…
274   GBEF01001042.1.p1  GBEF01001042.1.p1 GENE.GBEF01001042.1~~GBEF010…
9162  GBEF01044251.1.p1  GBEF01044251.1.p1 GENE.GBEF01044251.1~~GBEF010…
```

3222	GBEF01023315.1.p1	GBEF01023315.1.p1	GENE.GBEF01023315.1~~GBEF010…
9161	GBEF01044250.1.p1	GBEF01044250.1.p1	GENE.GBEF01044250.1~~GBEF010…
9163	GBEF01044252.1.p1	GBEF01044252.1.p1	GENE.GBEF01044252.1~~GBEF010…
2372	GBEF01017801.1.p1	GBEF01017801.1.p1	GENE.GBEF01017801.1~~GBEF010…
9158	GBEF01044247.1.p1	GBEF01044247.1.p1	GENE.GBEF01044247.1~~GBEF010…
4337	GBEF01025817.1.p1	GBEF01025817.1.p1	GENE.GBEF01025817.1~~GBEF010…
4338	GBEF01025818.1.p1	GBEF01025818.1.p1	GENE.GBEF01025818.1~~GBEF010…
9159	GBEF01044248.1.p1	GBEF01044248.1.p1	GENE.GBEF01044248.1~~GBEF010…
9238	GBEF01044458.1.p1	GBEF01044458.1.p1	GENE.GBEF01044458.1~~GBEF010…
4587	GBEF01026239.1.p2	GBEF01026239.1.p2	GENE.GBEF01026239.1~~GBEF010…
7823	GBEF01041323.1.p1	GBEF01041323.1.p1	GENE.GBEF01041323.1~~GBEF010…
1067	GBEF01015093.1.p1	GBEF01015093.1.p1	GENE.GBEF01015093.1~~GBEF010…
630	GBEF01002344.1.p1	GBEF01002344.1.p1	GENE.GBEF01002344.1~~GBEF010…
5996	GBEF01030153.1.p1	GBEF01030153.1.p1	GENE.GBEF01030153.1~~GBEF010…
6285	GBEF01031892.1.p1	GBEF01031892.1.p1	GENE.GBEF01031892.1~~GBEF010…
6287	GBEF01031894.1.p1	GBEF01031894.1.p1	GENE.GBEF01031894.1~~GBEF010…
7023	GBEF01034428.1.p1	GBEF01034428.1.p1	GENE.GBEF01034428.1~~GBEF010…
6284	GBEF01031891.1.p1	GBEF01031891.1.p1	GENE.GBEF01031891.1~~GBEF010…
6286	GBEF01031893.1.p1	GBEF01031893.1.p1	GENE.GBEF01031893.1~~GBEF010…
7022	GBEF01034427.1.p1	GBEF01034427.1.p1	GENE.GBEF01034427.1~~GBEF010…
5285	GBEF01028087.1.p1	GBEF01028087.1.p1	GENE.GBEF01028087.1~~GBEF010…
7149	GBEF01034741.1.p1	GBEF01034741.1.p1	GENE.GBEF01034741.1~~GBEF010…
632	GBEF01002345.1.p1	GBEF01002345.1.p1	GENE.GBEF01002345.1~~GBEF010…
9057	GBEF01044016.1.p1	GBEF01044016.1.p1	GENE.GBEF01044016.1~~GBEF010…
5287	GBEF01028088.1.p1	GBEF01028088.1.p1	GENE.GBEF01028088.1~~GBEF010…
6005	GBEF01030217.1.p1	GBEF01030217.1.p1	GENE.GBEF01030217.1~~GBEF010…
9241	GBEF01044460.1.p1	GBEF01044460.1.p1	GENE.GBEF01044460.1~~GBEF010…
8995	GBEF01043862.1.p2	GBEF01043862.1.p2	GENE.GBEF01043862.1~~GBEF010…
8989	GBEF01043856.1.p2	GBEF01043856.1.p2	GENE.GBEF01043856.1~~GBEF010…
9250	GBEF01044472.1.p1	GBEF01044472.1.p1	GENE.GBEF01044472.1~~GBEF010…
1608	GBEF01016312.1.p1	GBEF01016312.1.p1	GENE.GBEF01016312.1~~GBEF010…
4917	GBEF01027125.1.p1	GBEF01027125.1.p1	GENE.GBEF01027125.1~~GBEF010…
4918	GBEF01027126.1.p1	GBEF01027126.1.p1	GENE.GBEF01027126.1~~GBEF010…
6260	GBEF01031768.1.p1	GBEF01031768.1.p1	GENE.GBEF01031768.1~~GBEF010…
857	GBEF01006357.1.p1	GBEF01006357.1.p1	GENE.GBEF01006357.1~~GBEF010…
4586	GBEF01026239.1.p1	GBEF01026239.1.p1	GENE.GBEF01026239.1~~GBEF010…
858	GBEF01006358.1.p1	GBEF01006358.1.p1	GENE.GBEF01006358.1~~GBEF010…
7000	GBEF01034340.1.p2	GBEF01034340.1.p2	GENE.GBEF01034340.1~~GBEF010…
6998	GBEF01034339.1.p2	GBEF01034339.1.p2	GENE.GBEF01034339.1~~GBEF010…
7001	GBEF01034341.1.p1	GBEF01034341.1.p1	GENE.GBEF01034341.1~~GBEF010…
123	GBEF01000494.1.p1	GBEF01000494.1.p1	GENE.GBEF01000494.1~~GBEF010…
4919	GBEF01027127.1.p1	GBEF01027127.1.p1	GENE.GBEF01027127.1~~GBEF010…
8768	GBEF01043405.1.p1	GBEF01043405.1.p1	GENE.GBEF01043405.1~~GBEF010…
9236	GBEF01044457.1.p1	GBEF01044457.1.p1	GENE.GBEF01044457.1~~GBEF010…
149	GBEF01000572.1.p1	GBEF01000572.1.p1	GENE.GBEF01000572.1~~GBEF010…
5348	GBEF01028215.1.p1	GBEF01028215.1.p1	GENE.GBEF01028215.1~~GBEF010…
8769	GBEF01043406.1.p1	GBEF01043406.1.p1	GENE.GBEF01043406.1~~GBEF010…
8783	GBEF01043449.1.p1	GBEF01043449.1.p1	GENE.GBEF01043449.1~~GBEF010…

```
8624  GBEF01043062.1.p1  GBEF01043062.1.p1 GENE.GBEF01043062.1~~GBEF010…
8990  GBEF01043857.1.p1  GBEF01043857.1.p1 GENE.GBEF01043857.1~~GBEF010…
8996  GBEF01043863.1.p1  GBEF01043863.1.p1 GENE.GBEF01043863.1~~GBEF010…
```

	protein_length	probabilidade_resistencia
4488	892	0.999993
3221	872	0.999989
4489	866	0.999988
3223	874	0.996271
789	504	0.996198
3220	881	0.996154
9160	579	0.995670
274	867	0.994766
9162	532	0.993943
3222	883	0.992787
9161	572	0.992545
9163	525	0.989016
2372	611	0.989001
9158	713	0.986505
4337	886	0.979193
4338	886	0.979193
9159	706	0.978576
9238	851	0.974369
4587	408	0.971997
7823	311	0.962537
1067	1025	0.959229
630	359	0.958145
5996	430	0.955643
6285	246	0.947734
6287	246	0.947734
7023	1126	0.946059
6284	259	0.943942
6286	259	0.943942
7022	1128	0.936942
5285	293	0.934990
7149	417	0.931965
632	311	0.922010
9057	711	0.918803
5287	486	0.918170
6005	532	0.915668
9241	454	0.913889
8995	207	0.912767
8989	207	0.912767
9250	563	0.909116
1608	865	0.905514
4917	420	0.904884
4918	420	0.904884
6260	528	0.904830

```
857             553             0.900491
4586            414             0.890897
858             551             0.889836
7000            121             0.887609
6998            121             0.887609
7001            121             0.887609
123             483             0.884751
4919            423             0.883423
8768            921             0.882453
9236            585             0.880821
149             544             0.877514
5348            321             0.876449
8769            915             0.872643
8783            501             0.870331
8624            482             0.866370
8990            495             0.865208
8996            370             0.851808

VALIDAÇÃO BIOLÓGICA

P_LOOP : 24
KINASE2 : 13
GLPL : 38
MHD : 21
RNBS_A : 0
RNBS_B : 31
RNBS_C : 41
RNBS_D : 48
HRD : 1
DFG : 1
LRR1 : 4
LRR2 : 12


Finalizado predição MELONGENA: 12:51:26

Tempo decorrido: 0:00:04.134238
```

[32]:
```
print("\nPredição para TUBEROSUM\n")
inicio = datetime.now()
print("\n\n\nInciando predição TUBEROSUM: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_tuberosum_fasta,␣
 ↪svm_model,"SVM",output="predicoes_tuberosum_svm.csv",planta="Solanum␣
 ↪tuberosum")
fim = datetime.now()
print("\n\nFinalizado predição TUBEROSUM:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)
```

```
Predição para TUBEROSUM



Inciando predição TUBEROSUM:  12:51:26

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/tuberosum_transcriptoma.fasta, MODELO: SVM
                      id                                                descricao  \
9242  XM_015312842.1.p1  XM_015312842.1.p1 GENE.XM_015312842.1~~XM_0153…
7736  XM_015304639.1.p1  XM_015304639.1.p1 GENE.XM_015304639.1~~XM_0153…
7737  XM_015304640.1.p1  XM_015304640.1.p1 GENE.XM_015304640.1~~XM_0153…
7738  XM_015304641.1.p1  XM_015304641.1.p1 GENE.XM_015304641.1~~XM_0153…
7739  XM_015304642.1.p1  XM_015304642.1.p1 GENE.XM_015304642.1~~XM_0153…
…                   …                                                   …
7685  XM_015304550.1.p2  XM_015304550.1.p2 GENE.XM_015304550.1~~XM_0153…
7687  XM_015304551.1.p1  XM_015304551.1.p1 GENE.XM_015304551.1~~XM_0153…
7599  XM_015304365.1.p1  XM_015304365.1.p1 GENE.XM_015304365.1~~XM_0153…
2462  XM_006357493.2.p1  XM_006357493.2.p1 GENE.XM_006357493.2~~XM_0063…
8099  XM_015305454.1.p1  XM_015305454.1.p1 GENE.XM_015305454.1~~XM_0153…

      protein_length  probabilidade_resistencia
9242            1216                   1.000000
7736            1023                   0.999999
7737            1271                   0.999999
7738            1271                   0.999999
7739            1267                   0.999999
…                  …                          …
7685             285                   0.856698
7687             680                   0.855640
7599             526                   0.855485
2462             922                   0.854650
8099             701                   0.853323

[425 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 310
KINASE2 : 169
GLPL : 332
MHD : 341
RNBS_A : 0
RNBS_B : 301
RNBS_C : 378
```

```
RNBS_D : 410
HRD : 3
DFG : 61
LRR1 : 92
LRR2 : 174


Finalizado predição TUBEROSUM: 12:51:31

Tempo decorrido: 0:00:04.814188
```

[33]:
```
print("\nPredição para PHYSALIS\n")
inicio = datetime.now()
print("\n\n\nInciando predição PHYSALIS: ", inicio.strftime("%H:%M:%S"))
predizer_transcritos(transcriptoma_physalis_fasta,␣
 ↪svm_model,"SVM",output="predicoes_physalis_svm.csv",planta="Physalis␣
 ↪peruviana")
fim = datetime.now()
print("\n\nFinalizado predição PHYSALIS:", fim.strftime("%H:%M:%S"))
print("\nTempo decorrido:", fim - inicio)

print("\nArquivo salvo:")
print(f"{OUTPUT_DIR}/predicoes_svm.csv")
```

```
Predição para PHYSALIS



Inciando predição PHYSALIS:  12:51:31

TOP CANDIDATOS com probabilidade de >=
0.85
pipelineAG/physalis_transcriptoma.fasta, MODELO: SVM
                       id                                                 descricao  \
10249  IABH01023929.1.p1  IABH01023929.1.p1 GENE.IABH01023929.1~~IABH010…
9950   IABH01023452.1.p1  IABH01023452.1.p1 GENE.IABH01023452.1~~IABH010…
10240  IABH01023909.1.p1  IABH01023909.1.p1 GENE.IABH01023909.1~~IABH010…
9618   IABH01022921.1.p1  IABH01022921.1.p1 GENE.IABH01022921.1~~IABH010…
9951   IABH01023453.1.p1  IABH01023453.1.p1 GENE.IABH01023453.1~~IABH010…
…                    …                                                  …
9585   IABH01022880.1.p1  IABH01022880.1.p1 GENE.IABH01022880.1~~IABH010…
7883   IABH01019924.1.p1  IABH01019924.1.p1 GENE.IABH01019924.1~~IABH010…
7885   IABH01019929.1.p1  IABH01019929.1.p1 GENE.IABH01019929.1~~IABH010…
1130   IABH01004440.1.p1  IABH01004440.1.p1 GENE.IABH01004440.1~~IABH010…
1708   IABH01006528.1.p1  IABH01006528.1.p1 GENE.IABH01006528.1~~IABH010…
```

```
       protein_length  probabilidade_resistencia
10249             530                   1.000000
9950              423                   0.999999
10240             849                   0.999999
9618              873                   0.999999
9951              387                   0.999999
…                 …                      …
9585             1034                   0.852415
7883              469                   0.851678
7885              469                   0.851678
1130              226                   0.851652
1708              645                   0.851378

[130 rows x 4 columns]

VALIDAÇÃO BIOLÓGICA

P_LOOP : 63
KINASE2 : 30
GLPL : 94
MHD : 87
RNBS_A : 0
RNBS_B : 91
RNBS_C : 112
RNBS_D : 118
HRD : 2
DFG : 6
LRR1 : 13
LRR2 : 43


Finalizado predição PHYSALIS: 12:51:36

Tempo decorrido: 0:00:04.942692

Arquivo salvo:
pipelineAG/predicoes_svm.csv
```

[34]:
```
print(df_validacao)
```

```
[{'P_LOOP': 231, 'KINASE2': 31, 'GLPL': 222, 'MHD': 219, 'RNBS_A': 0, 'RNBS_B':
243, 'RNBS_C': 297, 'RNBS_D': 303, 'HRD': 13, 'DFG': 66, 'LRR1': 68, 'LRR2':
130, 'planta': 'Arabidopsis thaliana', 'modelo': 'XGBoost'}, {'P_LOOP': 46,
'KINASE2': 14, 'GLPL': 60, 'MHD': 41, 'RNBS_A': 0, 'RNBS_B': 54, 'RNBS_C': 80,
'RNBS_D': 82, 'HRD': 3, 'DFG': 22, 'LRR1': 7, 'LRR2': 21, 'planta': 'Solanum
melongena', 'modelo': 'XGBoost'}, {'P_LOOP': 335, 'KINASE2': 180, 'GLPL': 376,
'MHD': 375, 'RNBS_A': 0, 'RNBS_B': 350, 'RNBS_C': 437, 'RNBS_D': 487, 'HRD': 19,
'DFG': 86, 'LRR1': 100, 'LRR2': 190, 'planta': 'Solanum tuberosum', 'modelo':
```

```
'XGBoost'}, {'P_LOOP': 94, 'KINASE2': 31, 'GLPL': 114, 'MHD': 115, 'RNBS_A': 0,
'RNBS_B': 116, 'RNBS_C': 149, 'RNBS_D': 161, 'HRD': 7, 'DFG': 10, 'LRR1': 11,
'LRR2': 48, 'planta': 'Physalis peruviana', 'modelo': 'XGBoost'}, {'P_LOOP':
143, 'KINASE2': 27, 'GLPL': 140, 'MHD': 126, 'RNBS_A': 0, 'RNBS_B': 134,
'RNBS_C': 142, 'RNBS_D': 143, 'HRD': 7, 'DFG': 38, 'LRR1': 33, 'LRR2': 86,
'planta': 'Arabidopsis thaliana', 'modelo': 'RF'}, {'P_LOOP': 12, 'KINASE2': 12,
'GLPL': 12, 'MHD': 7, 'RNBS_A': 0, 'RNBS_B': 3, 'RNBS_C': 8, 'RNBS_D': 12,
'HRD': 1, 'DFG': 0, 'LRR1': 4, 'LRR2': 7, 'planta': 'Solanum melongena',
'modelo': 'RF'}, {'P_LOOP': 251, 'KINASE2': 163, 'GLPL': 243, 'MHD': 243,
'RNBS_A': 0, 'RNBS_B': 217, 'RNBS_C': 246, 'RNBS_D': 260, 'HRD': 2, 'DFG': 52,
'LRR1': 76, 'LRR2': 124, 'planta': 'Solanum tuberosum', 'modelo': 'RF'},
{'P_LOOP': 44, 'KINASE2': 29, 'GLPL': 44, 'MHD': 49, 'RNBS_A': 0, 'RNBS_B': 47,
'RNBS_C': 49, 'RNBS_D': 50, 'HRD': 0, 'DFG': 5, 'LRR1': 6, 'LRR2': 26, 'planta':
'Physalis peruviana', 'modelo': 'RF'}, {'P_LOOP': 170, 'KINASE2': 27, 'GLPL':
167, 'MHD': 148, 'RNBS_A': 0, 'RNBS_B': 183, 'RNBS_C': 211, 'RNBS_D': 214,
'HRD': 7, 'DFG': 43, 'LRR1': 39, 'LRR2': 99, 'planta': 'Arabidopsis thaliana',
'modelo': 'SVM'}, {'P_LOOP': 24, 'KINASE2': 13, 'GLPL': 38, 'MHD': 21, 'RNBS_A':
0, 'RNBS_B': 31, 'RNBS_C': 41, 'RNBS_D': 48, 'HRD': 1, 'DFG': 1, 'LRR1': 4,
'LRR2': 12, 'planta': 'Solanum melongena', 'modelo': 'SVM'}, {'P_LOOP': 310,
'KINASE2': 169, 'GLPL': 332, 'MHD': 341, 'RNBS_A': 0, 'RNBS_B': 301, 'RNBS_C':
378, 'RNBS_D': 410, 'HRD': 3, 'DFG': 61, 'LRR1': 92, 'LRR2': 174, 'planta':
'Solanum tuberosum', 'modelo': 'SVM'}, {'P_LOOP': 63, 'KINASE2': 30, 'GLPL': 94,
'MHD': 87, 'RNBS_A': 0, 'RNBS_B': 91, 'RNBS_C': 112, 'RNBS_D': 118, 'HRD': 2,
'DFG': 6, 'LRR1': 13, 'LRR2': 43, 'planta': 'Physalis peruviana', 'modelo':
'SVM'}]
```

[35]:
```
df_copia = df_validacao.copy()
df_inicial = pd.DataFrame(df_validacao)

df_validacao = df_inicial.melt(
    id_vars=["planta", "modelo"],
    var_name="motivo",
    value_name="contagem"
)
print(df_validacao.head())

df_validacao.to_csv(f"{OUTPUT_DIR}/df_validacao.csv", index=False)
```

```
                planta   modelo  motivo  contagem
0  Arabidopsis thaliana  XGBoost  P_LOOP       231
1     Solanum melongena  XGBoost  P_LOOP        46
2     Solanum tuberosum  XGBoost  P_LOOP       335
3    Physalis peruviana  XGBoost  P_LOOP        94
4  Arabidopsis thaliana       RF  P_LOOP       143
```

[36]:
```
#Scatterplot
plt.figure(figsize=(12,6))
sns.scatterplot(
```

```
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="planta",
    style="modelo",
    s=200
)
# plt.title( "Validação biológica dos motivos conservados")
plt.ylabel("Número de ocorrências")
plt.xlabel("Motivo")
plt.legend(
    bbox_to_anchor=(1.05,1),
    loc="upper left"
)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/Validacao_biologica_dos_motivos_conservados.png")
plt.close()
# plt.show()

#Bubble Chart
plt.figure(figsize=(12,6))
sns.scatterplot(
    data=df_validacao,
    x="motivo",
    y="modelo",
    hue="planta",
    size="contagem",
    sizes=(50,600)
)
# plt.title("Distribuição dos motivos por modelo e espécie")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/distribuicao_dos_motivos_por_modelo_especie.png")
plt.close()
# plt.show()


#Facetas por planta
g = sns.catplot(
    data=df_validacao,
    x="motivo",
    y="contagem",
    hue="modelo",
    col="planta",
    kind="bar",
    height=5,
    aspect=1.2
)
```

```
g.set_xticklabels(rotation=45)
plt.savefig(f"{OUTPUT_DIR}/facetas_por_planta.png")
plt.close()
# plt.show()

#por planta

#Physalis peruviana
df_physalis = df_validacao[df_validacao["planta"] == "Physalis peruviana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_physalis,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Physalis peruviana", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_physalis.png", dpi=300)
plt.close()

#Solanum melongena
df_melongena = df_validacao[df_validacao["planta"] == "Solanum melongena"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_melongena,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
```

```
)

# plt.title("Comparação dos Modelos para Solanum melongena", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_melongena.png", dpi=300)
plt.close()

#Solanum tuberosum
df_tuberosum = df_validacao[df_validacao["planta"] == "Solanum tuberosum"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_tuberosum,
    x="motivo",
    y="contagem",
    hue="modelo",
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Solanum tuberosum", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_tuberosum.png", dpi=300)
plt.close()

#Arabidopsis thaliana
df_arabidopsis = df_validacao[df_validacao["planta"] == "Arabidopsis thaliana"]
plt.figure(figsize=(12, 6))
sns.set_theme(style="whitegrid")
sns.lineplot(
    data=df_arabidopsis,
    x="motivo",
    y="contagem",
    hue="modelo",
```

```
    style="modelo",
    markers=True,      # Ativa os marcadores nos pontos
    dashes=False,      # Mantém as linhas contínuas
    linewidth=2,       # Espessura da linha
    markersize=10      # Tamanho dos pontos de dispersão
)

# plt.title("Comparação dos Modelos para Arabidopsis thaliana", fontsize=14,␣
 ↪fontweight="bold", pad=15)
plt.xlabel("Motivos Conservados", fontsize=12, labelpad=10)
plt.ylabel("Número de Ocorrências (Contagem)", fontsize=12, labelpad=10)
plt.xticks(rotation=45, ha="right")
plt.legend(title="Modelos", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/comparacao_modelos_arabidopsis.png", dpi=300)
plt.close()

#Heatmap
df_validacao["grupo"] = (
    df_validacao["planta"]
    + "_"
    + df_validacao["modelo"]
)

pivot = df_validacao.pivot_table(
    index="motivo",
    columns="grupo",
    values="contagem",
    fill_value=0
)


plt.figure(figsize=(14, 8))
sns.heatmap(
    pivot,
    annot=True,
    fmt="g",
    cmap="YlGnBu",
    linewidths=0.5,
    cbar_kws={'label': 'Contagem'} # Nomeia a barra de cores lateral
)

# plt.title("Distribuição dos Motivos Conservados por Planta e Modelo",␣
 ↪fontsize=14, fontweight="bold", pad=15)
plt.ylabel("Motivo", fontsize=12)
plt.xlabel("Grupo (Planta & Modelo)", fontsize=12)
plt.xticks(rotation=45, ha="right", fontsize=10)
```

```
plt.yticks(rotation=0, fontsize=10)
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/heatmap_distribuicao_dos_motivos_conservados.
  ↪png",dpi=300)
plt.close()
# plt.show()


print("========= Fim do Pipeline ============")
```

```
========= Fim do Pipeline ============
```